

DISCIPLINA DE QUALIDADE DE SOFTWARE



Sumário

<i>UNIDADE I – Os Modelos de Referência de Qualidade de Software</i>	3
O modelo de referência MPS/BR	3
O modelo de referência CMMI	11
Normas ISO.....	14
Considerações Finais.....	16
<i>UNIDADE II – A Qualidade do Processo e do Produto</i>	17
Qualidade do produto	17
Qualidade do processo.....	21
Processos de Gerência de Qualidade de Software	26
Considerações Finais.....	27
<i>UNIDADE III – Garantia da Qualidade de Software</i>	29
Garantia de qualidade de software	29
Verificação e validação.....	31
Revisões e auditorias	34
Considerações Finais.....	39
<i>UNIDADE IV – Requisitos de Qualidade de Software</i>	40
Considerações práticas	40
Requisitos de qualidade de software	42
Caracterização de defeitos	56
Considerações Finais.....	60
<i>UNIDADE V – Aspectos Operacionais de Qualidade de Software</i>	61
Técnicas de gerência de qualidade de software	61
Medidas de qualidade de software	62
Ferramentas de Qualidade de Software	67
Considerações Finais.....	70
<i>Referências</i>	72

Apresentação

Everton Gomedes

Doutorando em Engenharia Elétrica/Computação pelo Programa de Pós-graduação da Faculdade de Engenharia Elétrica e de Computação (FEEC) da Universidade Estadual de Campinas (UNICAMP). Mestre em Ciência da Computação pela Universidade Estadual de Londrina. MBA (Master of Business Administration) em Neurogestão pela Fundação Getúlio Vargas. MBA em Gerenciamento de Projetos pela FGV. Especialista em Engenharia de Software e Banco de Dados pela Universidade Estadual de Londrina. Graduado em Processamento de Dados. Sun Certified Programmer for Java 2 Platform 1.4 e ITIL® Foundation Certificate in IT Service Management. Experiência de mais de 10 anos na área de TI em arquitetura de software, engenharia de software, mapeamento de processos de negócio, gerenciamento de projetos, desenvolvimento de sistemas transacionais e de suporte à decisão. Experiência em projetos nacionais e internacionais em instituições financeiras e empresas de desenvolvimento de software. Professor de cursos de pós-graduação em universidades públicas e privadas. Membro do PMI 1580489. Publicações nacionais e internacionais. Proficiência em inglês atestada pelo TOIEC e TOEFL. Áreas de Interesse: Estrutura de Dados e Organização de Arquivos, Análise de Sistemas e Engenharia de Software, Lógica de Programação, Redes de Computadores, Linguagem de Programação, Administração de Banco de Dados, Governança de TIC (ITIL/CobIT), Data Warehouse, Data Mining, Big Data, Gerenciamento de Projetos, Gerenciamento de Processos de Negócio (BPM/BPMN™), Pesquisa Operacional, Algoritmos, Sistemas de Suporte à Decisão, Data Science e Machine Learning.

Ricardo Satin

Mestre em Engenharia de Software pela UTFPR; Especialista em Computação pela Unicesumar; Graduado em Computação pela Universidade Estadual de Maringá -UEM; MBA em Gerenciamento de Projetos pela FGV. Profissional certificado em: gerenciamento de projeto pelo P.M.I. (profissional PMP); Gerenciamento ágil de projetos (SCRUM); Gerenciamento de serviços de T.I. (ITIL). Professor de ensino superior para cursos da área de T.I. e pós-graduação para diversas áreas. Atua como consultor em gerenciamento de projetos; processos de negócio; melhoria de processos e machine learning.

UNIDADE I – Os Modelos de Referência de Qualidade de Software

Objetivos de aprendizagem

- O modelo de referência MPS/BR
- O modelo de referência CMMI
- Normas ISO

Quando falamos em qualidade geralmente vem a nossa cabeça a imagem de produtos caros e sofisticados que, no geral, são de acesso impossível para grande parte das pessoas. Essa imagem não poderia estar mais errada. Uma das possíveis definições de qualidade está associada com adequação aos requisitos (Pressman, 2011). Esta definição, apesar de simples, fornece uma visão abrangente e precisa do que vem a ser qualidade. Vamos aplicar tal definição a software. Um software que está adequado a seus requisitos funcionais tem qualidade? Sim. Que está adequado aos requisitos não funcionais, tem qualidade? Sim. Dessa forma, vamos manter essa definição ao longo desse estudo.

O modelo de referência MPS/BR

Uma das maneiras de medir o quanto um software tem de qualidade pode ser dado com relação ao processo que o produz. Dessa maneira, se o processo produtor do software possui qualidade, existe uma grande chance de o software ter qualidade. Se o processo produtor do software não tem qualidade, existe uma grande probabilidade de o software ser de baixa qualidade também. Neste caso, existe um esforço relacionado com a padronização do que vem a ser qualidade.

Neste cenário, um dos modelos brasileiros mais conhecido é o MPS.BR criado e mantido pela Associação para Promoção da Excelência do Software Brasileiro (Softex) apoiado pelo Ministério da Ciência e Tecnologia (MCT). Este modelo possui como diretrizes a (1) a implantação de engenharia de softwares de forma adequada ao contexto das empresas brasileiras e (2) compatibilidade com os padrões de qualidade aceitos internacionalmente. Possui 7 níveis de maturidade, compatibilidade com o CMMI, ISO 12.207 e 15.504. Tem como foco as micro, pequenas e médias empresas brasileiras.

O modelo de qualidade possui 3 componentes: (1) modelo de referência, (2) modelo de avaliação e (3) modelo de negócio. O MPS possui ainda 4 guias: (1) guia geral, (2) guia de implementação, (3) guia de aquisição e (4) guia de avaliação. A três primeiras guias são ligadas ao componente modelo de referência e a última guia citada, como o próprio nome sugere, é ligada ao modelo de avaliação, formando, então, a seguinte estrutura:

- Modelo de Referência: guias geral, de implementação e de avaliação
- Modelo de Avaliação: guia de avaliação
- Modelo de Negócio: documentos do programa

Esse modelo define também os níveis de maturidade que uma organização pode atingir ao criar software. Tais níveis estão descritos na Figura 1, abaixo:

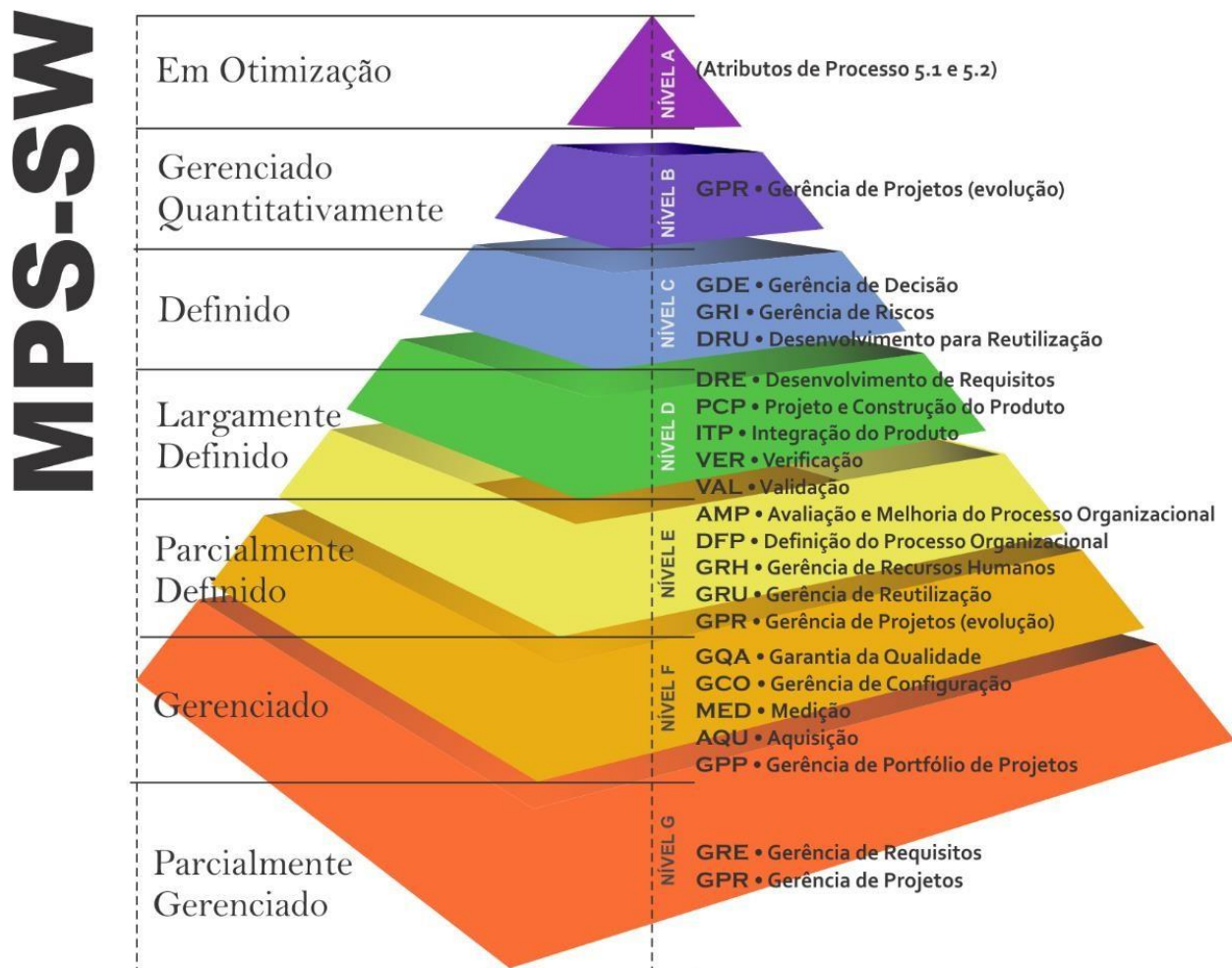


Figura 1 – Níveis de Maturidade de Organizações Produtoras de Software (MPS.BR)

Para obter a certificação em cada nível é necessário o atingimento de um conjunto de requisitos determinados para cada um deles. Mesmo que a organização não tenha a necessidade de certificação, compreender os elementos necessários para cada nível é importante e ajuda a estruturar o processo de engenharia de software para que o mesmo aumente a chances de produzir software de qualidade e evitar o que é conhecido como “programador herói”.

#REFLITA#

Programador herói é um termo utilizado pelo profissional que tenta manter um software construído com poucas práticas de engenharia de software. Esse profissional acaba sendo sobrecarregado e produz poucas coisas úteis pois está sempre tentando corrigir defeitos.
#REFLITA#

A base técnica do MR-MPS-SW

O Modelo de Referência MR-MPS-SW (MPS para Software) tem como base técnica a NBR ISO/IEC 12.207:2009 Engenharia de sistemas e software – Processos de ciclo de vida de software e o CMMI-DEV®. Este modelo está descrito no Guia Geral MPS de Software. A ISO 12.207 define uma estrutura e terminologias comuns para processos de ciclo de vida de software. A estrutura contém processos, atividades e tarefas que serão realizados durante o fornecimento, desenvolvimento, operação, manutenção e descontinuidade dos produtos de software, assim como durante a aquisição de um produto de software ou serviço.

O CMMI® (*Capability Maturity Model® Integration* – Modelo Integrado de Maturidade e de Capacidade) é um modelo de maturidade para melhoria de processo, administrado pelo CMMI Institute®, para o desenvolvimento de produtos e serviços. Este modelo define as melhores práticas associadas as atividades de desenvolvimento e de manutenção do ciclo de vida do produto, desde a concepção até a entrega e a manutenção.

Os Níveis de Maturidade do MR-MPS-SW

O MR-MPS-SW é o modelo de referência de software orientado a níveis de maturidade, que são uma combinação entre Processos e sua Capacidade. O Processo é definido por seu propósito e seus resultados esperados. O resultado esperado do processo é atendido por meio de atividades e tarefas. Cada nível do Modelo é atendido por um conjunto de processos a serem implementados. A Capacidade do Processo é a caracterização da habilidade do processo para alcançar os objetivos de negócio. Cada nível do Modelo é atendido por um conjunto de atributos de processo associados. O MR-MPS-SW possui 7 níveis de maturidade, apresentado abaixo do inicial ao final:

- Nível G: Parcialmente Gerenciado
- Nível F: Gerenciado
- Nível E: Parcialmente Definido
- Nível D: Largamente Definido
- Nível C: Definido
- Nível B: Gerenciado Quantitativamente
- Nível A: Em otimização

A seguir vamos estudar para cada nível, os propósitos dos processos, bem como os resultados esperados e atributos conforme definido no Guia Geral MPS de Software.

Nível G: Parcialmente Gerenciado

O nível de maturidade G é o primeiro nível a ser implementado do modelo. Este nível é composto por dois processos, com os seguintes propósitos e resultados esperados:

- Gerência de Projetos (GPR): O propósito é estabelecer e manter planos que determinam as atividades, recursos e responsabilidades do projeto, bem como prover informações sobre o andamento do projeto que permitam a realização de correções quando houver desalinhamentos significativos no desempenho do projeto. Devem ser implementados 19 resultados esperados, neste nível.
- Gerência de Requisitos (GRE): O intuito é gerenciar os requisitos do produto e dos componentes do produto do projeto e identificar inconsistências entre os requisitos, os planos do projeto e os produtos de trabalho do projeto. Devem ser implementados 5 resultados esperados.

Neste nível a implementação dos processos deve atender aos seguintes atributos de processo:

- AP 1.1: O processo é executado. O objetivo é medir o quanto o propósito do processo é alcançado pela sua execução.
- AP 2.1: A execução do processo é gerenciada. O objetivo é medir o quanto a execução do processo é gerenciada.

O nível G é o nível inicial. Este nível fornece as bases para os demais níveis e é um dos mais importantes pois inicia a mudança de cultura na organização que está disposta a implementá-lo.

Nível F: Gerenciado

O nível de maturidade F é o segundo nível do modelo, sendo aderente ao o nível 2 do CMMI-DEV. Este nível é composto pelos processos do nível anterior (GPR e GRE) e por mais cinco processos, com os seguintes propósitos e resultados esperados:

- Aquisição (AQU): O propósito é gerenciar a aquisição de produtos que satisfaçam às necessidades expressas pelo adquirente. Devem ser implementados 8 resultados esperados.
- Gerência de Configuração (GCO): O propósito é estabelecer e manter a integridade de todos os produtos de trabalho de um processo ou projeto e disponibilizá-los a todos os envolvidos. Devem ser implementados 7 resultados esperados.

- Garantia da Qualidade (GQA): O propósito é assegurar que os produtos de trabalho e a execução dos processos estejam em conformidade com os planos, procedimentos e padrões estabelecidos. Devem ser implementados 4 resultados esperados.
- Gerência de Portfólio de Projetos (GPP): O propósito é iniciar e manter projetos que sejam necessários, suficientes e sustentáveis, de forma a atender os objetivos estratégicos da organização. Devem ser implementados 8 resultados esperados.
- Medição (MED): O propósito é coletar, armazenar, analisar e relatar os dados relativos aos produtos desenvolvidos e aos processos implementados na organização e em seus projetos, de forma a apoiar os objetivos organizacionais. Devem ser implementados 7 resultados esperados.

Neste nível a implementação dos processos deve atender aos atributos de processo dos níveis anteriores e ao seguinte atributo de processo:

- AP 2.2: Os produtos de trabalho do processo são gerenciados. O objetivo é medir o quanto os produtos de trabalho do processo são gerenciados, isto é, produzidos, controlados e mantidos.

Este nível tem uma importância para a implementação. Ele faz com que o esforço de gerenciamento seja conhecido e com os resultados medidos. Fundamenta as bases para o gerenciamento dos projetos de software.

Nível E: Parcialmente Definido

No nível de maturidade E entram os processos organizacionais. É composto pelos processos dos níveis anteriores (GPR, GRE, AQU, GCO, GQA, GPP e MED), pelos resultados evoluídos do Gerência de Projetos (3 resultados esperados) e por mais quatro processos, com os seguintes propósitos e resultados esperados:

- Avaliação e Melhoria do Processo Organizacional (AMP): O propósito é determinar o quanto os processos padrão da organização contribuem para alcançar os objetivos de negócio da organização e para apoiar a organização a planejar, realizar e implantar melhorias contínuas nos processos com base no entendimento de seus pontos fortes e fracos. Devem ser implementados 10 resultados esperados.
- Definição do Processo Organizacional (DFP): O propósito é estabelecer e manter um conjunto de ativos de processo organizacional e padrões do ambiente de trabalho usáveis e aplicáveis às necessidades de negócio da organização. Devem ser implementados 8 resultados esperados.
- Gerência de Recursos Humanos (GRH): O propósito é prover a organização e os projetos com os recursos humanos necessários e manter suas competências

adequadas às necessidades do negócio. Devem ser implementados 11 resultados esperados.

- Gerência de Reutilização (GRU): O propósito é gerenciar o ciclo de vida dos ativos reutilizáveis. Devem ser implementados 5 resultados esperados.

Neste nível a implementação dos processos deve atender aos atributos de processo dos níveis anteriores e os seguintes atributos de processo:

- AP 3.1: O processo é definido. O objetivo é medir o quanto o processo padrão da organização é mantido de forma a apoiar sua adaptação para um processo definido.
- AP 3.2: O processo está implementado. O objetivo é medir o quanto o processo padrão está implementado na organização.

Os processos definidos neste nível envolvem atividades da organização que influenciam a forma como a mesma produz software. Por exemplo, temos o GRH que está diretamente ligado ao que pode se chamar de recursos humanos de uma empresa.

Nível D: Largamente Definido

O nível de maturidade D entram os processos da engenharia de software. É composto pelos processos dos níveis anteriores (GPR, GRE, AQU, GCO, GQA, GPP, MED, AMP, DFP, GRH, GRU) e por mais cinco processos, com os seguintes propósitos e resultados esperados:

- Desenvolvimento de Requisitos (DRE): O propósito é definir os requisitos do cliente, do produto e dos componentes do produto. Devem ser implementados 7 resultados esperados.
- Integração do Produto (ITP): O propósito é compor os componentes do produto, produzindo um produto integrado consistente com seu projeto, e demonstrar que os requisitos funcionais e não-funcionais são satisfeitos para o ambiente alvo ou equivalente. Devem ser implementados 9 resultados esperados.
- Projeto e Construção do Produto (PCP): O propósito é projetar, desenvolver e implementar soluções para atender aos requisitos. Devem ser implementados 8 resultados esperados.
- Validação (VAL): O propósito é confirmar que um produto ou componente do produto atenderá a seu uso pretendido quando colocado no ambiente para o qual foi desenvolvido. Devem ser implementados 7 resultados esperados.
- Verificação (VER): O propósito é confirmar que cada serviço e/ou produto de trabalho do processo ou do projeto atende apropriadamente os requisitos especificados. Devem ser implementados 6 resultados esperados.

Neste nível a implementação dos processos deve atender aos atributos de processo dos níveis anteriores. Adicionalmente, aqui temos a validação e verificação que ajuda muito na qualidade do produto. Vamos ver mais detalhes sobre esse assunto mais a frente.

Nível C: Definido

O nível de maturidade C é composto pelos processos dos níveis de maturidade anteriores (GPR, GRE, AQU, GCO, GQA, GPP, MED, AMP, DFP, GRH, GRU, DRE, ITP, PCP, VAL, VER), por mais três processos, com os seguintes propósitos e resultados esperados:

- Desenvolvimento para Reutilização (DRU): O propósito é identificar oportunidades de reutilização sistemática de ativos na organização e, se possível, estabelecer um programa de reutilização para desenvolver ativos a partir de engenharia de domínios de aplicação. Devem ser implementados 9 resultados esperados.
- Gerência de Decisões (GDE): O propósito é analisar possíveis decisões críticas usando um processo formal, com critérios estabelecidos, para avaliação das alternativas identificadas. Devem ser implementados 7 resultados esperados.
- Gerência de Riscos (GRI): O propósito é identificar, analisar, tratar, monitorar e reduzir continuamente os riscos em nível organizacional e de projeto. Devem ser implementados 9 resultados esperados.

Neste nível a implementação dos processos deve satisfazer os atributos dos níveis anteriores. Um ponto importante desse nível tem relação medir as decisões. Isso permite com que a governança possa ser melhorada por conta desse processo.

#REFLITA#

O quanto a forma como as decisões são formuladas auxiliam na qualidade do software.

#REFLITA#

Nível B: Gerenciado Quantitativamente

O nível de maturidade C é composto pelos processos dos níveis de maturidade anteriores (GPR, GRE, AQU, GCO, GQA, GPP, MED, AMP, DFP, GRH, GRU, DRE, ITP, PCP, VAL, VER) e pelos resultados evoluídos do Gerência de Projetos (7 resultados esperados). Neste nível a implementação dos processos deve atender aos atributos de processo dos níveis anteriores e aos seguintes atributos de processo:

- AP 4.1: O processo é objeto de análise quantitativa. O objetivo é medir o quanto as necessidades de informação são definidas, os relacionamentos entre os elementos de processo são identificados e dados são coletados.

- AP 4.2: O processo é controlado quantitativamente. O objetivo é medir o quanto dados objetivos são utilizados para gerenciar o desempenho do processo que é previsível.

Neste nível o gerenciamento passa a ser quantitativo (de verdade). Isso não é algo tão simples de ser implementado.

Nível A: Em Otimização

O nível de maturidade A é composto pelos processos dos níveis de maturidade anteriores (G ao B). Não possui processos específicos do nível. Neste nível a implementação dos processos deve atender aos atributos de processo dos níveis anteriores e aos seguintes atributos de processo:

- AP 5.1: O processo é objeto de melhorias incrementais e inovações. O objetivo é medir o quanto mudanças no processo são identificadas a partir de investigação de enfoques inovadores para a definição e implantação do processo.
- AP 5.2: O processo é objeto de implementação de melhorias inovadoras e incrementais. O objetivo é medir o quanto as mudanças na definição, gerência e desempenho do processo alcançou os objetivos.

O Modelo de Referência MR-MPS-SW está sendo implementado desde 2003, com inúmeros casos de sucesso. Este modelo é compatível com o CMMI-DEV (*Capability Maturity Model Integration*) e capacita a Organização para a internacionalização. Seu custo benefício é excelente para Empresas que desejam se tornar competitivas no mercado interno e externo. Porém, as empresas enfrentam também vários fracassos na implementação do MR-MPS-SW. Geralmente, esses fracassos estão associados a uma implementação *by the book*¹ que ao invés de agregar valor gera burocracias sem necessidade emperrando o processo. Um bom *roadmap* de implantação é iniciar com o nível G, que estrutura a Empresa e já implementa os conceitos de gerenciamento de projetos e requisitos. As Empresas iniciam o pensamento sistêmico de transformar requisitos em processos e atividades, preparando a base da pirâmide da maturidade organizacional.

#REFLITA#

O modelo de referência deve ser utilizado como uma caixa de ferramentas. Ele deve ser adaptado a sua realidade e não ser seguido do começo ao fim. Isso aumenta as chances do mesmo ser aceito em sua organização. Além disso, permita que esse modelo possa ser alterado sempre que necessário, mesmo depois de implementado.

#REFLITA#

¹ Sem mudar nada do modelo. Isso geralmente causa a falha na implementação

O modelo de referência CMMI

O CMMI (*Capability Maturity Model Integration* ou Modelo Integrado de Maturidade em Capacitação) é um modelo de referência que contém práticas necessárias à maturidade em disciplinas específicas. Desenvolvido pelo SEI (*Software Engineering Institute*) da Universidade Carnegie Mellon, o CMMI é uma evolução do CMM e procura estabelecer um modelo único para o processo de melhoria corporativo, integrando diferentes modelos e disciplinas.

O CMMI foi baseado nas melhores práticas para desenvolvimento e manutenção de produtos. Há uma ênfase tanto em engenharia de sistemas quanto em engenharia de software, e há uma integração necessária para o desenvolvimento e a manutenção. A versão atual do CMMI (versão 1.3) foi publicada em 27 de outubro de 2010 e apresenta três modelos:

- *CMMI for Development* (CMMI-DEV), voltado ao processo de desenvolvimento de produtos e serviços.
- *CMMI for Acquisition* (CMMI-ACQ), voltado aos processos de aquisição e terceirização de bens e serviços.
- *CMMI for Services* (CMMI-SVC), voltado aos processos de empresas prestadoras de serviços.

Uma das premissas do modelo é "A qualidade é influenciada pelo processo", e seu foco é "Melhorar processo de uma empresa". O modelo CMMI v1.2 (CMMI-DEV) contém 22 áreas de processo. Esse modelo é a versão internacional para desenvolvimento de software. Esse modelo costuma ser adotado no Brasil também, especialmente quando a empresa é exportadora de software. Esse modelo serve de referência em outros países e de uma certa forma, nivela o assunto para organizações em países diferentes. Em sua representação por estágios, as áreas são divididas da seguinte forma:

- Nível 1: Inicial(Ad-hoc)
 - Não possui áreas de processo
- Nível 2: Gerenciado
 - Gerenciamento de Requisitos
 - Planejamento de Projeto
 - Acompanhamento e Controle de Projeto
 - Gerenciamento de Acordo com Fornecedor
 - Medição e Análise
 - Garantia da Qualidade de Processo e Produto

- Gerência de Configuração
- Nível 3: Definido
 - Desenvolvimento de Requisitos
 - Solução Técnica
 - Integração de Produto
 - Verificação
 - Validação
 - Foco de Processo Organizacional
 - Definição de Processo Organizacional
 - Treinamento Organizacional
 - Gerenciamento Integrado de Projeto
 - Gerenciamento de Riscos
 - Análise de Decisão e Resolução
- Nível 4: Quantitativamente gerenciado
 - Desempenho de Processo Organizacional
 - Gerenciamento Quantitativo de Projeto
- Nível 5: Otimização
 - Gestão do Desempenho Organizacional
 - Análise Causal e Resolução

Um processo representa, dentro da área de software, um conjunto de atividades cujo objetivo é atingir uma meta previamente estipulada. Já por capacidade e maturidade de um processo, deve-se ter a noção do grau de qualidade com o qual um processo atinge um resultado esperado. Dentre os principais benefícios da implantação do CMMI, vale a pena destacar:

- Uma maior confiabilidade no que refere ao cumprimento de prazos e custos que foram acordados, inicialmente, perante o cliente que solicitou o desenvolvimento de um sistema. Essa previsibilidade é decorrente do rigor que o CMMI exige quanto à medição dos processos, fato este que conduz à obtenção de uma base histórica realista e confiável para estes fins
- O gerenciamento das atividades relativas à produção de software aumenta consideravelmente
- Uma maior qualidade nos softwares criados, já que processos bem definidos e controlados conduzem à produção de produtos mais confiáveis

- A menor dependência da empresa de desenvolvimento para com seus especialistas. Com um foco voltado para processos e melhoria contínua, além do uso intensivo de informações históricas, a organização deixa de depender única e exclusivamente de profissionais com um elevado grau de conhecimento técnico
- A busca por melhorias contínuas nos processos cotidianos

Para se conseguir o que este modelo propõe, a organização interessada na implantação do CMMI deverá evoluir progressivamente, considerando para isto uma sucessão de diferentes de níveis. Cada nível indica, por sua vez, o grau de maturidade dos processos num determinado instante:

- No nível 1 os processos normalmente estão envoltos num caos decorrente da não-obediência ou ainda, inexistência de padrões
- No nível 2 os projetos têm seus requisitos gerenciados neste ponto. Além disso, há o planejamento, a medição e o controle dos diferentes processos
- No nível 3 os processos já estão claramente definidos e são compreendidos dentro da organização. Os procedimentos se encontram padronizados, além de ser preciso prever sua aplicação em diferentes projetos
- No nível 4 ocorre o aumento da previsibilidade do desempenho de diferentes processos, uma vez que os mesmos já são controlados quantitativamente
- No nível 5 existe uma melhoria contínua dos processos

A implantação do CMMI é aconselhável para grandes organizações de software. Implementar os diversos estágios é uma tarefa complexa, não só numa fase inicial, mas também quando se leva em conta a migração de um nível para outro. Isto exigirá, sistematicamente a realização de grandiosos investimentos financeiros, assim como uma mudança de postura da organização. Em inúmeras ocasiões, empresas desenvolvedoras de sistemas recorrem a consultorias especializadas, visando ajuda na obtenção da certificação CMMI. Outro ponto importante é que a implementação do modelo muda a forma como a empresa produz software e conseqüentemente o software produzido.

#ATENÇÃO#

Caso tenha interesse em consultar as empresas no Brasil que possuam algum nível de certificação CMMI, basta acessar o endereço abaixo.

<https://sas.cmmiinstitute.com/pars/pars.aspx>

#ATENÇÃO#

Normas ISO

A ISO/IEC 9126 é uma norma ISO para qualidade de *produto de software*². Ela define um conjunto de parâmetros com o objetivo de padronizar a avaliação da qualidade de software. Ela se enquadra no modelo de qualidade das normas da família 9000. A ISO/IEC 9126, sob o título geral "Engenharia de software - Qualidade do produto", consiste nas seguintes partes:

- Parte 1: Modelo de qualidade
- Parte 2: Métricas externas
- Parte 3: Métricas internas
- Parte 4: Métricas de qualidade em uso

Segundo a NBR ISO 9000:2005, "*qualidade é o grau no qual um conjunto de características inerentes satisfaz aos requisitos*". Ou seja, pode-se afirmar que se algum produto ou serviço atende aos requisitos especificados, este mesmo produto ou serviço possui a qualidade desejada. A qualidade pode ser medida por meio do grau de satisfação em que as pessoas avaliam determinado produto ou serviço. No entanto, esse produto ou serviço pode ter qualidade para algumas pessoas e para outras nem tanto, ou seja, a qualidade é algo subjetivo.

Alguns modelos de qualidade de software também são citados por Pressman (Pressman, 2017). Conhecido como Fatores da Qualidade, estes fatores avaliam o software em três pontos distintos: Transição do Produto, Revisão do Produto e Operação do Produto. Na Figura 2 são mostrados os Fatores da Qualidade de McCall:



Figura 2 – Os Fatores da Qualidade de McCall (CHRISSIS, 2006)

Assim como o modelo proposto por McCall (CHRISSIS, 2006) a Hewlett-Packard desenvolveu também um modelo que referencia fatores da qualidade de software e

² Existe uma discussão sobre se software é produto ou serviço. Tal definição ainda não é clara e vem sendo debatida entre os especialistas do assunto.

denominado FURPS: *Functionality, Usability, Reliability, Performance e Supportability*. Estes fatores estabelecem as métricas de qualidade de software para cada fase do processo de engenharia de software. Além desses modelos de métricas para qualidade de software, nota-se que a constante busca pela mesma se tornou uma atividade essencial dentro das empresas. Colocando-se todos esses conceitos dentro do contexto apresentado, podemos dizer que qualidade não é uma fase do ciclo de desenvolvimento de software, mas sim parte de todas as fases (Pressman, 2011).

Portanto, é necessário um planejamento adequado para que a qualidade de software seja atingida, conforme a definição de qualidade que deverá ser alcançada. Para isso são necessários modelos, padrões, métodos e técnicas para alcançar essas metas de qualidade propostas. Para tanto, todas as etapas do ciclo de vida de engenharia de software devem ser contempladas com atividades que visam garantir a qualidade tanto do processo quanto do produto. O PMBOK (*Project Management Body of knowledge*) do PMI (*Project Management Institute*), na sua versão 2017 (PMI, 2017) apresenta o gerenciamento da qualidade do projeto, conforme Figura 3 abaixo:

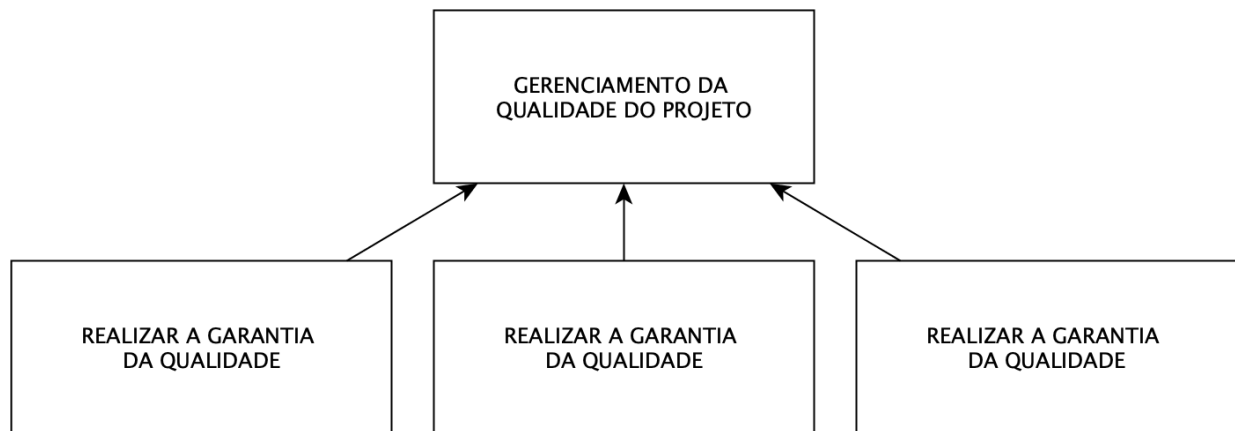


Figura 3 – Controle de Qualidade Segundo o PMI (PMI, 2017)

De acordo com o PMBOK, os processos de gerenciamento da qualidade do projeto incluem todas as atividades da organização executora que definem as responsabilidades, os objetivos e as políticas de qualidade, de modo que o projeto atenda às necessidades que motivaram sua realização. Eles implementam o sistema de gerenciamento da qualidade por meio da política, dos procedimentos e dos processos de planejamento da qualidade, garantia da qualidade e controle da qualidade, com atividades de melhoria contínua dos processos conduzidas do início ao fim, conforme adequado. Com isso os três principais processos são:

- Planejamento da Qualidade: Identificação dos padrões de qualidade relevantes para o projeto e determinação de como satisfazê-los
- Garantia da Qualidade: Aplicação das atividades de qualidade planejadas e sistemáticas para garantir que o projeto emprega todos os processos necessários para atender aos requisitos

- Controle da Qualidade: Monitoramento de resultados específicos do projeto a fim de determinar se eles estão de acordo com os padrões relevantes de qualidade e identificação de maneiras de eliminar as causas de um desempenho insatisfatório

Como se pode notar é evidente a semelhança entre os conceitos usados no PMBOK e os conceitos da própria ISO. Com isso, é possível ainda relacionar estes três processos do PMBOK com as definições de controle da qualidade e garantia da qualidade de software apresentados anteriormente.

Considerações Finais

A qualidade de um produto de software era algo intangível, sem definições concretas. A norma em questão permite visualizar mais facilmente esta qualidade e, por consequência, definir uma forma de medir esta qualidade, possibilitando uma avaliação mais objetiva e uniforme. Além disso, é acessível a qualquer pessoa ou empresa, disponível a quem se interessar. Por meio das características abordadas, em todo o processo em que o software é englobado, são permitidas sub características e métricas, aprofundando aos principais atributos do software, o que esperar e o que atingir. Além disso, existem algumas normas relacionadas ao assunto. Dentre elas, podemos citar:

- ISO 9126 e ISO 25010 – Qualidade de produto de software
- ISO/IEC 12119 – Essa norma é aplicada aos pacotes de softwares, como por exemplo: bancos de dados, software gráficos, entre outros
- ISO/IEC 14598-5 – Esta parte da ABNT NBR ISO/IEC 14598 fornece requisitos e recomendações para implementação prática de avaliação de um produto de software, quando várias partes envolvidas necessitam entender, aceitar e confiar nos resultados da avaliação. Em especial, ela pode ser usada na aplicação dos conceitos descritos na ISO IEC 9126
- ISO/IEC 12207 – A ISO/IEC 12207 é a norma ISO/IEC que define processo de Engenharia de Software, atividades e tarefas que são associados com os processos do ciclo de vida do software desde sua concepção até a retirada/descontinuação do software

Os modelos de referência, sejam eles CMMI, MPS ou ISOs, devem ser usados como base. Evite a comparação entre eles ou misturar todos em um processo de engenharia de software orientado a qualidade.

UNIDADE II – A Qualidade do Processo e do Produto

Objetivos de aprendizagem

- Qualidade do produto
- Qualidade do processo
- Processos de gerência de qualidade de software

A qualidade pode ser encarada como um espectro que envolve vários elementos para a produção de software. Esse espectro precisa de conhecido para evitar a confusão e para que sua aplicação seja efetiva e com o menor esforço possível. Neste estudo vamos analisar o espectro da qualidade de software.

Qualidade do produto

A norma brasileira correspondente a qualidade do produto é a NBR 13596, que foi substituída pela NBR ISO/IEC 9126-1. A norma 9126-1 foca na *qualidade do produto de software*, propondo atributos de qualidade, distribuídos em seis características principais, com cada uma delas divididas em sub-características, conforme podemos ver na Figura 4 abaixo:

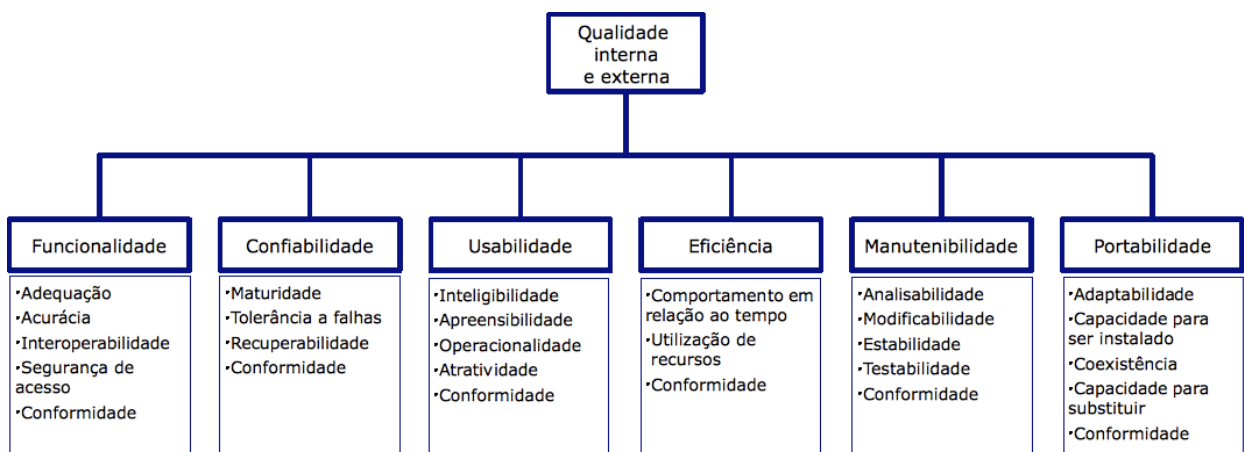


Figura 4 – Qualidade do Produto de Software

No nível mais alto temos as características de qualidade e nos quadros abaixo as suas sub-características. Cada característica/sub-características compõe um atributo de qualidade do software. Note que em todas as características temos uma subcategoria com o nome de Conformidade. A conformidade é utilizada para avaliar o quanto o software obedece aos requisitos de legislação e todo o tipo de padronização ou normalização aplicável ao contexto.

Funcionalidade

Capacidade de um software prover funcionalidades que satisfaçam o usuário em suas necessidades declaradas e implícitas, dentro de um determinado contexto de uso. Suas sub-características são:

- Adequação: capacidade do produto de software de prover um conjunto apropriado de funções para tarefas e objetivos do usuário especificados
- Acurácia: capacidade do produto de software de prover, com o grau de precisão necessário, resultados ou efeitos corretos ou conforme acordados
- Interoperabilidade: capacidade do produto de software de interagir com um ou mais sistemas especificados
- Segurança: capacidade do produto de software de proteger informações e dados, de forma que pessoas ou sistemas não autorizados não possam lê-los nem modificá-los e que não seja negado o acesso às pessoas ou sistemas autorizados

Confiabilidade

A capacidade do produto de software de manter um nível de desempenho especificado, quando usado em condições especificadas. Suas sub-características são:

- Maturidade: capacidade do produto de software de evitar falhas decorrentes de defeitos no software
- Tolerância a Falhas: capacidade do produto de software de manter um nível de desempenho especificado em casos de defeitos no software ou de violação de sua interface especificada
- Recuperabilidade: capacidade do produto de software de restabelecer seu nível de desempenho especificado e recuperar os dados diretamente afetados no caso de uma falha

Confiabilidade

A capacidade do produto de software de ser compreendido, aprendido, operado e atraente ao usuário, quando usado sob condições especificadas. Note que este conceito é bastante abrangente e se aplica mesmo a programas que não possuem uma interface para o

usuário final. Por exemplo, um programa *batch* executado por uma ferramenta de programação de processos também pode ser avaliado quanto a sua usabilidade, no que diz respeito a ser facilmente compreendido, aprendido, etc. Além disto, a operação de um sistema é uma Interface Humano-Computador (ver IHC) sujeita às avaliações de usabilidade. Suas sub-características são:

- Inteligibilidade: capacidade do produto de software de possibilitar ao usuário compreender se o software é apropriado e como ele pode ser usado para tarefas e condições de uso específicas
- Apreensibilidade: capacidade do produto de software de possibilitar ao usuário aprender sua aplicação
- Operacionalidade: capacidade do produto de software de possibilitar ao usuário operá-lo e controlá-lo
- Proteção frente a erros de usuários: como produto consegue prevenir erros dos usuários
- Estética/Atratividade: capacidade do produto atrair um potencial usuário para o sistema, o que pode incluir desde a adequação das informações prestadas para o usuário até os requintes visuais utilizados na sua interface gráfica
- Acessibilidade: refere-se a prática inclusiva de fazer softwares que possam ser utilizados por todas as pessoas que tenham deficiência ou não. Quando os softwares são corretamente concebidos, desenvolvidos e editados, todos os usuários podem ter igual acesso à informação e funcionalidades

Eficiência

O tempo de execução e os recursos envolvidos são compatíveis com o nível de desempenho do software. Suas sub-características são:

- Comportamento em Relação ao Tempo: que avalia se os tempos de resposta (ou de processamento) estão dentro das especificações
- Utilização de Recursos: que mede tanto os recursos consumidos quanto a capacidade do sistema em utilizar os recursos disponíveis; exemplo: processador e memória

Manutenibilidade

A capacidade (ou facilidade) do produto de software ser modificado, incluindo tanto as melhorias ou extensões de funcionalidade quanto às correções de defeitos, falhas ou erros. Suas

sub-características são:

- Analisabilidade identifica a facilidade em se diagnosticar eventuais problemas e identificar as causas das deficiências ou falhas
- Modificabilidade caracteriza a facilidade com que o comportamento do software pode ser modificado
- Estabilidade avalia a capacidade do software de evitar efeitos colaterais decorrentes de modificações introduzidas
- Testabilidade representa a capacidade de se testar o sistema modificado, tanto quanto as novas funcionalidades quanto as não afetadas diretamente pela modificação

Portabilidade

A capacidade do sistema ser transferido de um ambiente para outro. Como "ambiente", devemos considerar todos os fatores de adaptação, tais como diferentes condições de infraestrutura (sistemas operacionais, versões de bancos de dados, etc.), diferentes tipos e recursos de hardware (tal como aproveitar um número maior de processadores ou memória). Além destes, fatores como idioma ou a facilidade para se criar ambientes de testes devem ser considerados como características de portabilidade. Suas sub-características são:

- Adaptabilidade, representando a capacidade do software se adaptar a diferentes ambientes sem a necessidade de ações adicionais
- Capacidade para ser Instalado identifica a facilidade com que pode se instalar o sistema em um novo ambiente
- Coexistência mede o quão facilmente um software convive com outros instalados no mesmo ambiente
- Capacidade para Substituir representa a capacidade que o sistema tem de substituir outro sistema especificado

Qualidade em Uso

Os atributos de qualidade em uso são categorizados em quatro características: eficácia, produtividade, segurança e satisfação.

- Qualidade em uso: capacidade do produto de software de permitir que usuários

especificados atinjam metas especificadas com eficácia, produtividade, segurança e satisfação em contextos de uso especificados

- Eficácia: capacidade do produto de software de permitir que usuários atinjam metas especificadas com acurácia e completude, em um contexto de uso especificado
- Produtividade: capacidade do produto de software de permitir que seus usuários empreguem quantidade apropriada de recursos em relação à eficácia obtida, em um contexto de uso especificado
- Segurança: capacidade do produto de software de apresentar níveis aceitáveis de riscos de danos a pessoas, negócios, software, propriedades ou ao ambiente, em um contexto de uso especificado
- Satisfação: capacidade do produto de software de satisfazer usuários, em um contexto de uso especificado

Qualidade do processo

Antes de iniciarmos esse assunto, um ponto importante a ser definido é o que é um processo. Segundo o IEEE, um processo é uma sequência de passos realizados para um determinado propósito. Adicionalmente, um processo de software é um conjunto de atividades, métodos, práticas e tecnologias que as pessoas utilizam para desenvolver e manter software e produtos relacionados (CMMI). E a definição que vamos utilizar nesse estudo: processo de software consiste em uma série de atividades, práticas, eventos, ferramentas e métodos que garantem, técnica e administrativamente que o software pode ser desenvolvido com qualidade e de maneira organizada, disciplinada e previsível. Dessa forma, podemos classificar um processo de software como imaturo, quando:

- Adhoc: processo improvisado por profissionais e gerências
- Não é rigorosamente seguido e o cumprimento não é controlado
- Altamente dependente dos profissionais atuais
- Baixa visão do progresso e da qualidade
- A qualidade do produto pode ficar comprometida para que prazos sejam cumpridos
- Arriscado do ponto de vista do uso de nova tecnologia
- Custos de manutenção excessivos
- Qualidade difícil de se prever

De maneira semelhante, um processo de software maduro pode ser classificado como:

- Coerente com as linhas de ação, o trabalho é efetivamente concluído
- Definido, documentado e melhorando constantemente
 - Compreendido
 - Utilizado
 - Vivo e ativo
- Com o apoio visível da alta administração e outras gerências
- Bem controlado - fidelidade ao processo é objeto de auditoria e de controle
- São utilizadas medições do produto e do processo
- Uso disciplinado da tecnologia

O objetivo em se definir um processo de software é facilitar a produção de sistemas de alta qualidade, atingindo as necessidades dos usuários finais, dentro de um cronograma e um orçamento previsíveis. Um processo de software não pode ser definido de forma universal. Para ser eficaz e conduzir à construção de produtos de boa qualidade, um processo deve ser adequado às especificidades do projeto em questão. Deste modo, processos devem ser definidos caso a caso, considerando-se as características da aplicação (domínio do problema, tamanho, complexidade etc.), a tecnologia a ser adotada na sua construção (paradigma de desenvolvimento, linguagem de programação, mecanismo de persistência etc.), a organização onde o produto será desenvolvido e a equipe de desenvolvimento.

Há vários aspectos a serem considerados na definição de um processo de software. No centro da arquitetura de um processo de desenvolvimento estão as atividades-chave desse processo: análise e especificação de requisitos, projeto, implementação e testes, que são a base sobre a qual o processo de desenvolvimento deve ser construído. Entretanto, a definição de um processo envolve a escolha de um modelo de ciclo de vida, o detalhamento de suas macro atividades, a escolha de métodos, técnicas e roteiros para a sua realização e a definição de recursos e artefatos necessários e produzidos (Sommerville, 2011).

A escolha de um modelo de ciclo de vida é o ponto de partida para a definição de um processo de desenvolvimento de software (Pressman, 2011). Um modelo de ciclo de vida, geralmente, organiza as macro atividades básicas do processo, estabelecendo precedência e dependência entre as mesmas. Para a definição completa do processo, cada atividade descrita no modelo de ciclo de vida deve ser decomposta e para suas subatividades, devem ser associados métodos, técnicas, ferramentas e critérios de qualidade, entre outros, formando uma base sólida para o desenvolvimento.

Adicionalmente, outras atividades, tipicamente de cunho gerencial, devem ser definidas, entre elas atividade de gerência de projetos e de controle e garantia da qualidade. Os seguintes fatores influenciam a definição de um processo e, por conseguinte, a escolha do modelo de processo a ser usado como base: tipo de software (sistema de informação, sistema de tempo real), paradigma de desenvolvimento (estruturado, orientado a objetos ou outro), domínio da aplicação, tamanho e complexidade do sistema, estabilidade dos requisitos, características da equipe e assim por diante. Ainda que os processos tenham de ser definidos caso a caso, de

maneira geral, o ciclo de vida de um software envolve, pelo menos, as seguintes fases:

- **Planejamento:** o objetivo do planejamento de projeto é proporcionar uma estrutura que de possibilidade ao gerente fazer estimativas razoáveis de recursos, custos e prazos. Uma vez determinado o escopo de software, com os requisitos esboçados, uma proposta de desenvolvimento deve ser elaborada, isto é, um plano de projeto deve ser elaborado ajustando o processo a ser utilizado no desenvolvimento de software. À medida que o projeto progride, o planejamento deve ser detalhado e atualizado regularmente. Pelo menos ao final de cada uma das fases do desenvolvimento, o planejamento como um todo deve ser revisado e o planejamento da fase seguinte deve ser detalhado. O planejamento e o acompanhamento do progresso fazem parte do processo de gerência de projeto.
- **Análise e Especificação de Requisitos:** nesta fase, o processo de levantamento de requisitos é intensificado. O escopo deve ser refinado e os requisitos mais bem definidos. Para entender a natureza do software a ser construído, o engenheiro de software tem de compreender o domínio do problema, bem como a funcionalidade e o comportamento esperados. Uma vez capturados os requisitos do sistema a ser desenvolvido, estes devem ser modelados, avaliados e documentados. Uma parte vital desta fase é a construção de um modelo descrevendo o que o software tem de fazer (e não como fazê-lo).
- **Arquitetura/Projeto:** esta fase é responsável por incorporar requisitos tecnológicos aos requisitos essenciais do sistema, modelados na fase anterior e, portanto, requer que a plataforma de implementação seja conhecida. Basicamente, envolve duas grandes etapas: projeto da arquitetura do sistema e projeto detalhado. O objetivo da primeira etapa é definir a arquitetura geral do software, tendo por base o modelo construído na fase de análise de requisitos. Essa arquitetura deve descrever a estrutura de nível mais alto da aplicação e identificar seus principais componentes. O propósito do projeto detalhado é detalhar o projeto do software para cada componente identificado na etapa anterior. Os componentes de software devem ser sucessivamente refinados em níveis maiores de detalhamento, até que possam ser codificados e testados.
- **Implementação:** o projeto deve ser traduzido para uma forma passível de execução pela máquina. A fase de implementação realiza esta tarefa, isto é, cada unidade de software do projeto detalhado é implementada.
- **Testes:** inclui diversos níveis de testes, como por exemplo, teste de unidade, teste de integração e teste de sistema. Inicialmente, cada unidade de software implementada deve ser testada e os resultados documentados. A seguir, os diversos componentes devem ser integrados sucessivamente até se obter o sistema. Finalmente, o sistema como um todo deve ser testado.
- **Entrega e Implantação:** uma vez testado, o software deve ser colocado em produção.

Para tal, é necessário treinar os usuários, configurar o ambiente de produção e, eventualmente, converter bases de dados. O propósito desta fase é estabelecer que o software satisfaz os requisitos dos usuários. Isto é feito instalando o software e conduzindo testes de aceitação. Quando o software tiver demonstrado prover as capacidades requeridas, ele pode ser aceito e a operação iniciada.

- Operação: nesta fase, o software é utilizado pelos usuários no ambiente de produção. Uma coisa importante aqui é garantir a disponibilidade e o desempenho para que a utilização seja satisfatória.
- Manutenção: o software sofrerá mudanças após ter sido entregue para o usuário. Alterações ocorrerão porque erros foram encontrados, porque o software precisa ser adaptado para acomodar mudanças em seu ambiente externo, ou porque o cliente necessita de funcionalidade adicional ou aumento de desempenho. Muitas vezes, dependendo do tipo e porte da manutenção necessária, essa fase pode requerer a definição de um novo processo, onde cada uma das fases precedentes é reaplicada no contexto de um software existente ao invés de um novo. Os modelos de processo, de maneira geral, contemplam as fases Análise e Especificação de Requisitos, Projeto, Implementação, Testes e Entrega e Implantação. A escolha de um modelo de processo é fortemente dependente das características do projeto. Assim, é importante conhecer alguns modelos de ciclo de vida e em que situações são aplicáveis. Os principais modelos de ciclo de vida podem ser agrupados em três categorias principais: modelos sequenciais, modelos incrementais e modelos evolutivos.

A Figura 5 abaixo exhibe os processos utilizados para a produção de software de qualidade. Neste caso, esses processos foram agrupados para o contexto de sistemas e de software.

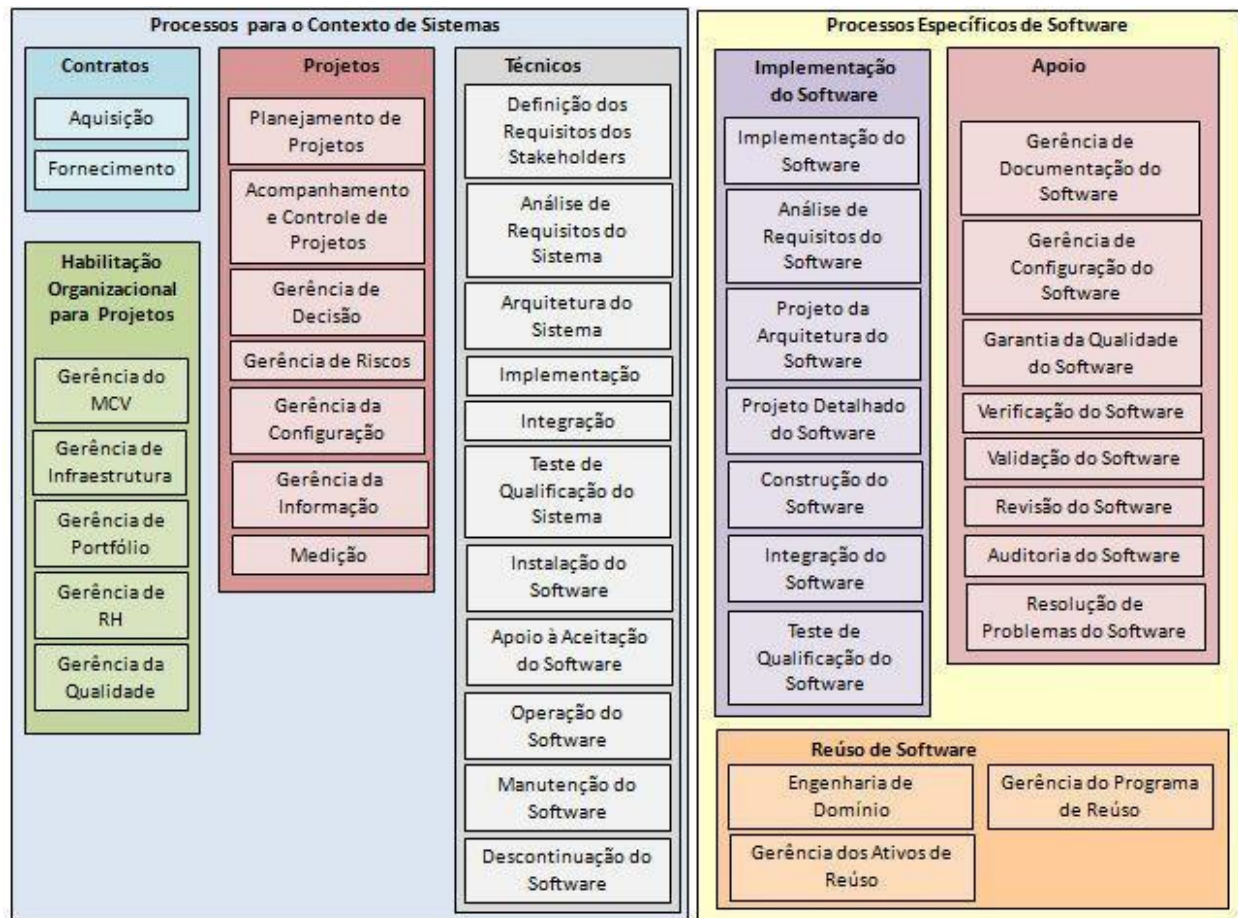


Figura 5 – Processos para a Qualidade de Software (Pressman, 2011)

Independente dos processos, existem 5 pontos que devem ser observados para aumentar a qualidade do software a ser produzido:

1. Alcance: deve ser capaz de lidar com várias tecnologias. A maioria dos aplicativos modernos contém vários idiomas e sistemas que são ligados entre si de forma complexa, com arquiteturas distribuídas
2. Tornar o conhecimento explícito de engenharia de software: deve ser capaz de verificar a aplicação inteira contra centenas de padrões de implementação que codificam as melhores práticas de engenharia
3. Métricas acionáveis: as métricas de qualidade não devem apenas informar, mas também orientar sobre como realizar a melhoria da qualidade do software, mostrando o que fazer primeiro, como fazê-lo, próximos passos etc. (esse é um ponto de automação)
4. Automação: finalmente, deve ser capaz de realizar todos os pontos descritos acima de forma automatizada. Nenhum profissional ou equipe pode fazer essa tarefa, muito menos fazê-la em um curto espaço de tempo

É importante medir a qualidade do software, mas é igualmente importante executar a atividade de forma correta. Essa ação é muito útil no desenvolvimento de software, mas, muitas vezes, é melhor não ter medição alguma do que contar com uma errada. Medição incorreta pode gerar uma ilusão de gerenciamento de qualidade.

Processos de Gerência de Qualidade de Software

A gerência de qualidade de software é a área responsável por garantir a eficiência dos produtos desenvolvidos. A preocupação com os produtos é uma característica de toda empresa competitiva e sustentável. Para aquelas que trabalham com desenvolvimento, a gerência de qualidade de software é a área que cuida disso. Se você quer desenvolver softwares e aplicativos com qualidade, esse assunto deve ser tratado em todas as fases do projeto e não apenas no fim.

Segundo a *International Standardization Organization* (ISO), qualidade é adequação ao uso e conformidade às exigências. Partindo desses princípios, a qualidade de software busca garantir que as aplicações desenvolvidas estejam de acordo com as necessidades do cliente e, ao mesmo tempo, atendam padrões e regras do desenvolvimento. Seguindo o raciocínio, a gerência de qualidade de software é a área que utiliza métodos de gestão para acompanhar o desenvolvimento do produto. Por exemplo, modelos de gerenciamento de projetos, como o *Project Management Body of Knowledge* (PMBOK), podem ser aplicados também em tecnologia. Os objetivos desses métodos de gestão quando aplicados à qualidade de software são:

- Garantir que o software atenda aos requisitos funcionais e de desempenho
- Assegurar que os processos de desenvolvimento estejam dentro dos padrões estabelecidos

Portanto, a gerência de qualidade de software procura evitar falhas como

- Erros de código, cálculo ou lógica
- Falta de planejamento das etapas de desenvolvimento
- Atrasos na entrega do produto
- Não prever e controlar riscos
- Não pensar na manutenção do produto

A gerência da qualidade de software trabalha como uma atividade guarda-chuva que permeia todo o ciclo de vida do software. Além disso, identifica e realiza modificações no processo quando necessário.

Considerações Finais

Neste estudo abordamos a qualidade do processo produtor do software e a qualidade do produto. Embora a qualidade do produto seja a percebida pelos usuários, a qualidade do processo é mais importante. Processos de qualidade geralmente produzem software de qualidade. Dados os conceitos de produto e processo, vale ressaltar a necessidade de a empresa diferenciar cada um desses conceitos. Uma vez que um produto esteja estabelecido no mercado ou no processo de fabricação, a natureza das mudanças e das inovações muda do foco no produto para foco no processo, havendo um esforço contínuo para a redução de custos, melhoria de desempenho, produtividade e qualidade. Algumas características que auxiliam na definição do foco da inovação para as empresas em termos de produtos e processos. São elas:

- Quanto mais padronizado e indiferenciado (*commodity*) for o produto, mais serão as inovações em processos
- Quanto mais lento o ciclo de vida tecnológico do produto, maior a taxa de inovações em processos
- As inovações em produtos são impulsionadas pela tecnologia e tendências de mercado
- As inovações em processos são impulsionadas por ideias para melhorias operacionais e redução de custos

Com relação aos produtos, a distinção entre produtos e processos é clara (por exemplo, no setor automobilístico em que o processo de desenvolvimento de um novo produto, como a introdução de um veículo, possui clara distinção quanto ao desenvolvimento do novo processo de produção do veículo). Para os serviços, porém, ela pode ser menos evidente, pois a produção, a distribuição e o consumo de muitos serviços podem ocorrer ao mesmo tempo. Algumas diretrizes diferenciadoras são:

- Se a inovação envolve características novas ou substancialmente melhoradas do serviço oferecido aos consumidores, trata-se de uma inovação de produto
- Se a inovação envolve métodos, equipamentos e/ou habilidades para o desempenho do serviço novo ou substancialmente melhorado, então é uma inovação de processo
- Se a inovação envolve melhorias substanciais nas características do serviço oferecido e nos métodos, equipamentos e/ou habilidades usados para seu desempenho, ela é uma inovação tanto de produto como de processo

Em muitos casos, uma inovação de serviço pode ser apenas de um tipo. Por exemplo, as

empresas podem oferecer um novo serviço ou novas características de um serviço, sem mudar substancialmente o método pelo qual ele é oferecido. Do mesmo modo, melhoramentos significativos em processos, por exemplo, a redução de custos de distribuição, podem não fazer qualquer diferença para as características do serviço vendido aos consumidores.

UNIDADE III – Garantia da Qualidade de Software

Objetivos de aprendizagem

- Garantia de qualidade de software
- Verificação e validação
- Revisões e auditorias

É melhor prevenir do que remediar. Esse ditado popular é a base para esse estudo onde a garantia da qualidade de software está relacionada com a prevenção de defeitos. A prevenção tende a ser mais barata do que a correção de defeitos, principalmente em operação.

Garantia de qualidade de software

A história do teste de software começa há muito tempo, mas foi em 1957 que o conceito de teste se tornou o processo de detecção de erros e não de verificar se o software funciona. Nesta época já se dizia que o teste deveria ser a intenção de encontrar erros (Christensen e Thayer, 2002). Com o passar dos anos, os projetos foram tendo mais e mais problemas devido à complexidade dos sistemas, segundo Christensen

“mais de 30% dos projetos são cancelados antes de serem finalizados e mais de 70% dos projetos falham nas entregas de funcionalidade”

Estas falhas criaram um ambiente caótico e problemático para os gestores, desta forma, criou-se a necessidade de implementação de processos de qualidade, que busquem estabelecer procedimentos que sirvam de garantia e gerenciamento de qualidade. Os testes devem provar que algo não funciona, e são essenciais para o sucesso do processo de qualidade de software.

“Os testes unitários podem resolver entre 30% a 50% dos defeitos dos programas, os testes de sistemas podem remover entre 30% a 50% dos defeitos remanescentes, desse modo, os sistemas podem ir para produção ainda com aproximadamente 49% dos defeitos. Por último, as revisões de códigos podem reduzir entre 30% e 30% desses defeitos.” (Chrissis e Konrad, 2006).

A realização de testes desde o início do desenvolvimento é algo aconselhado não só pelas metodologias ágeis, mas como pode ser visto em (Chrissis e Konrad, 2006), indicado para a garantia de qualidade. O processo de teste atua dentro de uma visão de qualidade como o PDCA³ no momento de *check*, mas em desenvolvimento, é possível realizar testes em todos os

³ Plan, Do, Check and Act

processos, do fazer (testes unitários) até o planejamento. A realização de testes de software pode ser feita de forma manual ou automatizada, esta última indica a utilização de ferramentas de teste. As ferramentas de teste permitem ao testador automatizar parte do processo de teste, facilitando a retestagem para validação dos testes.

“A atividade de teste, como a de desenvolvimento, gera grande número de informações e necessita de várias repetições, além de requerer coordenação e comunicação entre as equipes. As ferramentas de teste podem aliviar o fardo da produção e da execução de teste, da geração de informação e da comunicação.” (Christensen e Thayer, 2002)

Outro teste essencial para garantia dos requisitos não funcionais é o teste de performance e *stress*, que tem como função validar o limite da aplicação, entre eles o limite de usuários simultâneos, limite de dados do sistema, etc. Também são necessários os testes funcionais, que garantem que certas funcionalidades estão de acordo com os requisitos do projeto. Os testes funcionais podem ser exploratórios, manuais e automatizados.

Outro ponto fundamental nos dias atuais, principalmente depois que a Symantec apresentou o relatório que o Brasil é o 4º país com pior segurança de informação, são os testes de segurança. Existem diversas formas de analisar a segurança de um programa, verificando estouro de *buffer*, analisando ataques de SQL, entre outros. A qualidade de software está ligada a princípios que promovem a obtenção de satisfação dos usuários, da confiabilidade; do cumprimento do prazo estabelecido e do objetivo especificado, ou seja, das funcionalidades em perfeito funcionamento.

Além disso, revisões técnicas e testes para revelar e descobrir erros são realizadas. Como também, coleta e análise de erros são feitas para compreender como os erros são introduzidos e buscar eliminá-los. As mudanças, sejam em códigos ou no projeto, geram confusões. A qualidade de software (SQA) age fazendo uma análise desse aspecto com a finalidade de não ocasionar erros. A SQA tem por objetivo obter um produto final de alta qualidade. Sendo assim, tarefas são realizadas por um grupo para alcançar essa meta. Primeiramente, prepara-se um plano de SQA para um projeto; a equipe, então, participa no desenvolvimento da descrição de qualidade do projeto, revisa as atividades de engenharia de software e inspeciona softwares resultantes para verificar sua conformidade com a gestão da qualidade definida, garante, do mesmo modo, que o projeto seja documentado e registra qualquer problema para que seja resolvido.

As tarefas descritas, acima, servem para atingir qualidade dos requisitos, qualidade do projeto, qualidade do código e eficácia do controle de qualidade. Consideremos que a tecnologia atual faz-se presente em várias atividades diárias desempenhadas pelo ser humano. O avanço do software tem repercutido na sociedade de uma forma rápida, abrangente e profunda. E, a garantia da qualidade, está intimamente ligada, ao efeito que o software é capaz de proporcionar à sociedade. Exemplificando, um bug ou erro pode levar a óbito ou piorar o estado de pacientes, no caso, de projetos computacionais hospitalares. Sendo assim, quando se alcança a qualidade tem-se a confiabilidade de que as funcionalidades estarão sendo eficazes, funcionarão corretamente e falhas não serão encontradas ao ser usadas. Em suma, avaliar a

qualidade de um produto significa proceder a uma investigação científica minuciosa sobre a mesma. Implica em medir também, estatísticas de SQA. Consequentemente, a confiabilidade é rompida quando o projeto computacional começa a falhar frequentemente, falhas são associadas a problemas de projeto ou de implementação. Quando o usuário final percebe-as fica preocupado e perde a confiança que existia. A proteção do software é uma atividade que identifica e avalia problemas que provocam falhas.

Conclui-se, que a preocupação em construir softwares, vai além, de implementar códigos em computadores. Existe, necessidade dos elementos e tarefas citadas, anteriormente, para garantir a qualidade.

Verificação e validação

O objetivo da Validação e da Verificação (V&V) é assegurar que o software seja adequado e atende suas especificações. Durante este processo, podem ser utilizadas outras técnicas em conjunto, como a inspeção de software. A inspeção é a tarefa de analisar as representações do sistema, analisar os documentos de requisitos, diagramas e código-fonte. É recomendável a utilização da inspeção em todas as etapas do processo. Análise automatizada é uma técnica estática, a qual não se torna necessária à execução do software. Testes de software executam o sistema utilizando uma massa de dados. Esses testes servem para verificar se o desempenho do sistema está correto. O processo de V&V liga todo o ciclo de vida do sistema.

O teste de software é uma técnica dinâmica, a qual torna-se necessária a sua execução. Só é possível testar um software quando um protótipo ou uma versão executável do software está disponível. Esta é uma das vantagens do desenvolvimento incremental, a funcionalidade de cada parte do sistema pode ser testada, antes de serem acopladas as demais partes. Testar significa exercitar o programa, fazendo uso de dados reais. Estes serão processados pelo programa, e por meio da saída, poderão ser descobertos os defeitos, saídas incorretas e anomalias.

Inspeção de software é uma verificação estática, utilizada na descoberta de problemas. É uma revisão que pode ser aplicada a todos os artefatos. É um processo de detecção de defeitos bem definido. Após um defeito ser descoberto, é necessário corrigir o sistema e validá-lo. A partir daí, será necessário novas Inspeções no sistema, testes de regressões e análises. Para sistemas menores, um plano de testes com menos formalidade pode ser utilizado. Localizar e detectar defeitos em um programa, não é um processo simples. Às vezes é necessário projetar testes adicionais, os quais possam reproduzir os defeitos e localizarem estes. Ferramentas de *debugging* também ajudam na detecção de um problema.

#ATENÇÃO#

Durante uma Inspeção, pode ser necessário o uso de um checklist para melhor organização.

#ATENÇÃO#

As classes que serão verificadas no processo de inspeção, são: defeito de dados, controle, entrada e saída, interface, armazenamento e exceções. Desenvolvimento de software com um processo de qualidade tem a finalidade de evitar defeitos via inspeções. Esse processo é conhecido como *Cleanroom*⁴. Seu processo de desenvolvimento tem as seguintes etapas:

1. Especificar formalmente o sistema
2. Desenvolver perfil operacional
3. Definir incremento de software
4. Projetar testes estáticos
5. Construir programa estruturado
6. Verificar código
7. Integrar incremento
8. Testar sistemas integrados

Este processo é uma abordagem de desenvolvimento de software apoiado no desenvolvimento incremental e nos outros testes estáticos e dinâmicos.

#REFLITA#

Os tipos estático e dinâmico podem ser utilizados em conjunto? Devem ser utilizados em momentos diferentes? Sempre utilize esses conceitos como uma caixa de ferramenta que deve ser consultada conforme a necessidade do trabalho. Evite a abordagem de que uma ferramenta serve para todos os trabalhos.

#REFLITA#

Processo de verificação

O guia MPS.BR atribui a verificação ao nível D de maturidade. De acordo com o guia, o propósito da verificação consiste na confirmação do serviço ou processo de um trabalho do processo ou do projeto em conformidade com os requisitos especificados. Os resultados esperados abrangem a identificação do produto de trabalho a ser verificado, o desenvolvimento e a implementação de uma estratégia de verificação, identificação de critérios e procedimentos para verificação dos produtos de verificação e definição de um ambiente de verificação, execução de revisões por pares e testes, identificação e registro de defeitos e análise dos resultados e redirecionamento para as partes interessadas.

O MPS.BR descreve ainda a capacidade dos processos associados ao nível de Verificação, por meio de atributos do processo. Dessa forma, o processo do nível D é executado com geração de resultados bem definidos (AP 1.1), é executado com gerenciado (AP 2.1), possui produtos de trabalhos gerenciados (AP 2.2), é definido (AP 3.1) e está implementado (AP 3.2). O nível D,

⁴ Cleanroom é uma metodologia muito utilizada no desenvolvimento de software. É considerada "pesada" pelos padrões da Engenharia de Software, mas muito difundida no desenvolvimento de grandes projetos corporativos.

Largamente Definido, portanto, indica que a Verificação no contexto do MPS.BR é um processo bem definido, que perpassa uma série de passos e revisões. O processo de verificação definido pelo MPS.BR precisa ser definido, executado sob gerenciamento e com geração de resultados bem definidos, está implementado, e possui geração de resultados bem definidos. O processo é detalhado pelo guia em 6 tarefas gerais:

- VER 1: Identificar os produtos de trabalho a serem verificados
- VER 2: Desenvolver e implementar uma estratégia de verificação, com definição de um cronograma, revisores envolvidos, métodos para verificação e qualquer material a ser utilizado na verificação
- VER 3: Identificar critérios e procedimentos para verificação dos produtos de trabalho, além da definição do ambiente de verificação
- VER 4: Executar revisão por pares e testes e atividades de verificação
- VER 5: Identificar e registrar defeitos
- VER 6: Analisar os resultados gerados e encaminhar aos envolvidos

O processo de validação

O processo de Validação está incluído no MPS.Br no mesmo nível em que o processo de Verificação, o nível D, Largamente Definido. O propósito do processo de validação confirmar que um produto ou componente do produto atenderá a seu uso pretendido quando colocado no ambiente para o qual foi desenvolvido. Os atributos de processos são semelhantes ao processo de Verificação, uma vez que todos os processos de um mesmo nível possuem as capacidades definidas por ele. Portanto, o processo é definido, executado sob gerenciamento e com geração de resultados bem definidos, está implementado, e possui geração de resultados bem definidos (SOFTEX, 2011).

As atividades dentro do processo de validação abrangem a identificação dos produtos a serem avaliados, desenvolvimento e implementação de estratégias de avaliação, identificação de critérios e procedimentos para avaliação de produtos, execução das atividades de validação, identificação e registro de problemas, análise de resultados e encaminhamento aos envolvidos, evidências de que o produto está pronto para uso.

O processo de validação definido pelo MPS.BR, conforme, precisa ser definido, executado sob gerenciamento e com geração de resultados bem definidos, está implementado, e possui geração de resultados bem definidos. O processo é detalhado pelo guia, em 7 tarefas gerais, como resultados esperados:

- VAL 1: Identificar os produtos de trabalho a serem validados
- VAL 2: Desenvolver e implementar uma estratégia de validação, com definição de um

cronograma, revisores envolvidos, métodos para validação e qualquer material a ser utilizado na validação

- VAL 3: Identificar critérios e procedimentos para validação dos produtos de trabalho, além da definição do ambiente de validação
- VAL 4: Executar atividades de validação para garantir que o produto esteja pronto para ser disponibilizado em ambiente de uso
- VAL 5: Identificar e registrar problemas
- VAL 6: Analisar os resultados obtidos e encaminhar aos envolvidos
- VAL 7: Obter evidências de que os produtos de software desenvolvidos estão prontos para o uso pretendido são fornecidas

As evidências de que o produto de software está pronto para uso pode ser um requisito de *Compliance* (auditoria) que algumas organizações tem necessidade. Isso é comum na indústria financeira e médica.

Revisões e auditorias

Como todo padrão elaborado pelo IEEE⁵ (Instituto de Engenheiros Eletricistas e Eletrônicos), o 1028 é fruto do trabalho voluntário de alguns membros do IEEE. E sendo um padrão, eles nos traz algumas importantes e relevantes informações a respeito de revisão de software. Mas é sempre bom lembrar, que ele deve ser usado com bom senso, pois o contexto sempre prevalece sob o padrão. O IEEE 1028 nos traz cinco tipos de revisão de software, junto com os procedimentos necessários para a execução de cada tipo. Está fora do escopo do padrão questões como: quando uma revisão se faz necessária? como escolher qual tipo de revisão deve ser usado? Os 5 tipos de revisão abordados podem ser vistos na Tabela 1, abaixo.

⁵ Instituto de Engenheiros Eletricistas e Eletrônicos é uma organização profissional sem fins lucrativos, fundada nos Estados Unidos. É a maior organização profissional do mundo dedicada ao avanço da tecnologia em benefício da humanidade

Tabela 1. Os Cinco tipos de revisão (Christensen e Thayer, 2002)

Caract.	Gerencial	Técnica	Inspeção	Walkthrough	Auditoria
Objetivo	Garantir o progresso Recomendar ações corretivas Garantir alocação correta dos recursos	Avaliar a conformidade do estado atual com as especificações e planos garantir integridade da mudança	Encontrar anomalias verificar decisões verificar a qualidade do produto	Encontrar anomalias examinar alternativas melhorar o produto fórum para aprendizado	Avaliação independente de cumprimento com os objetivos de padrões e regulamentos
Tomada de decisão	A equipe de gerenciamento traça o curso da ação Decisões são feitas na reunião ou como resultado das recomendações	A equipe de revisão solicita aos gerentes ou a liderança técnica que atuem nas recomendações	A equipe de revisão escolhe as disposições pré-definidas do produto Os defeitos devem ser removidos	A equipe concorda com as mudanças para serem feitas pelo autor	Organização auditada, iniciador, comprador, cliente ou usuário
Verificação das mudanças	O líder verifica que itens são fechados A verificação das mudanças é deixada para outros controles do projeto	O líder verifica que itens são fechados A verificação das mudanças é deixada para outros controles do projeto	O líder verifica que itens são fechados A verificação das mudanças é deixada para outros controles do projeto	O líder verifica que itens são fechados A verificação das mudanças é deixada para outros controles do projeto	Responsabilidade da organização auditada
Tamanho recomendado do grupo	Duas ou mais pessoas	Três ou mais pessoas	Três a seis pessoas	Duas a sete pessoas	Uma a sete pessoas
Quem participa	Gerentes, liderança técnica e algumas pessoas de outras áreas	Liderança técnica e algumas pessoas de outras áreas	Pessoas da área com acompanhamento documentado	Liderança técnica e algumas pessoas de outras áreas	Auditores, organização auditada, pessoal de gerência e técnico
Grupo da liderança	Normalmente o gerente responsável	Normalmente o engenheiro líder	Um facilitador treinado	O facilitador ou o autor	O auditor líder
Volume de materiais	Moderado para muito, depende dos objetivos da reunião	Moderado para muito, depende dos objetivos da reunião	Relativamente baixo	Relativamente baixo	Moderado para muito, depende dos objetivos da reunião

É importante notar que a tomada de decisão precisa ser de conhecimento da equipe em todos os 5 tipos de revisão. Isso melhora a governança da empresa.

Revisão de Código x Programação em Par

O que seria *Revisão de Código*? Imagine a seguinte cena: você é um(a) jovem programador(a) em seu primeiro trabalho com desenvolvimento de software. Após terminar sua primeira tarefa, você é convocado(a) a uma sala de reunião; lá você encontra o desenvolvedor líder e três desenvolvedores seniores. Um monitor enorme está exibindo seu código. Durante duas horas e meia sofridas, seu código é julgado sem só enquanto você observa. Desde suas decisões de projeto até as mais triviais preferências estilísticas, nada escapa da enxurrada de críticas. É essa a cena exibida na sua mente quando você vê as palavras “revisão de código”? Este não é o tipo de revisão de código que estamos estudando. Ao invés, pense em um processo bem leve e informal. Você submete seu código para revisão criando um *pull request* ou mesmo usando alguma funcionalidade incorporada na sua IDE.

O objetivo é detectar bugs. Acredito que você é familiar com aquela sabedoria popular que diz que quanto mais tarde um defeito é encontrado, maior é o custo para sua resolução. Sendo assim, por que não usar um processo relativamente barato que pode encontrar até 60% dos defeitos? Outra razão para a sua empresa adotar revisões de código: melhorar a legibilidade do código. Tentar ler e entender um trecho novo de código frequentemente leva à descoberta de problemas como:

- Nomes ruins
- Identação errada
- Ocorrências de programação *cargo cult*⁶
- Código não idiomático

O revisor também pode encontrar questões que passaram despercebidas ou ajudar na avaliação de performance do código escrito. Adicionalmente, uma revisão de código bem feita pode espalhar conhecimento pelo time. Isso destrói ilhas de conhecimento na equipe, melhorando a qualidade final do produto.

Enquanto alguns dizem que a revisão de código é a prática mais importante que você deve adotar para aperfeiçoar seu código, tal entusiasmo não é universal. Assim, não poderíamos deixar de falar também das potenciais desvantagens da prática. A queixa mais comum sobre revisão de código tem a ver com tempo. Quando você envia algum código para revisão, você obviamente precisa esperar até que a revisão seja finalizada. O que fazer durante a espera? Idealmente, sua equipe deveria quebrar o trabalho disponível em unidades pequenas e discretas, que possam ser resolvidas de maneira mais ou menos independente. Nem sempre isso é prático, porém. E falando nisso, a própria troca constante de tarefas pode ser prejudicial para seu foco e produtividade.

⁶ Movimento religioso descrito pela primeira vez na Melanésia, que engloba uma série de práticas que ocorrem após o contato de sociedades simples com civilizações tecnologicamente mais avançadas

Digamos que o João gastou x horas implementando uma funcionalidade. Então a Ana revisa o trabalho dele e diz que a implementação está completamente errada, e ele precisa refazer tudo. Aquelas x horas de desenvolvimento acabaram de ir para o lixo. Finalmente, às vezes as pessoas desperdiçam quantidades ridículas de tempo discutindo sobre detalhes estéticos inúteis, como a posição das chaves, ou se devem ou não incluir um *underscore* no nome de um campo privado. Infelizmente, debates assim às vezes escalam para níveis que tornam o local de trabalho tóxico.

Podemos evitar isso com uma técnica chamada programação em par, que é uma técnica na qual duas pessoas colaboram no código juntas, na mesma estação de trabalho. Elas periodicamente revezam em dois papéis. O piloto escreve o código, pensando em voz alta para explicar suas decisões e o que está pensando. O navegador observa o piloto, dando feedback e opiniões em tempo real. Um dos princípios básicos dos métodos ágeis é que o desenvolvimento de software se beneficia de ciclos de feedback curtos. Levando isso em consideração, trazer a revisão de código para o mais cedo possível no processo faria bastante sentido, por estar em harmonia com os princípios ágeis.

Não deveria ser surpresa que a programação em par compartilha vários dos benefícios da revisão de código, como detecção de bugs, melhoras na legibilidade do código e propagação de conhecimento pela equipe. Mas a programação em par também pode oferecer benefícios exclusivos, tais como:

- Mais foco: A presença de um colega ao lado pode exercer uma certa pressão, lhe mantendo motivado na tarefa a ser realizada.
- Ciclos de feedback mais curtos: Já que seu par está revisando seu código o tempo todo, existe um risco muito menor de desperdiçar tempo graças a feedback tardio.
- Aumento no uso de outras boas práticas da Engenharia de Software: Como sugerido por um estudo realizado na Universidade Estadual da Carolina do Norte, equipes usando programação em par tendem a usar mais práticas de desenvolvimento como testes unitários, integração contínua e a criação de padrões de código.

Assim como no caso da revisão de código, programação em par está longe de ser uma prática universalmente aceita. Enquanto muitas pessoas utilizam a prática, outras não. Vamos começar com uma reclamação comum: programação em par pode ser extremamente cansativa. Na verdade, não é incomum ouvir que pairar é mais eficaz quando usado por blocos de tempo curtos — de 1 hora a 2 horas e meia.

Evidentemente, um número ímpar de integrantes na equipe não combina muito bem com programação em par. Mas um número flutuante de pessoal disponível é inevitável. O próximo item na nossa lista de problemas é o fato de que programação em par não é muito compatível com trabalho remoto. Você pode ter uma ideia melhor do problema após ler o que Daniel Kaplan, que escreveu *“What It’s Like to Pair for a Year”* tem a dizer sobre programação em par:

“Tais interrupções ocorrem, mas em um dia típico nós conseguimos evitá-las fazendo com que os pares cheguem na mesma hora (para café da manhã e reunião em pé), vão almoçar no mesmo horário, e vão para casa no mesmo horário. Isso maximiza o tempo que os pares passam programando juntos. Então, programação em par requer sincronismo, o que pode torná-la uma opção inviável para equipes remotas (ou mesmo equipes que trabalham no mesmo local mas tem horários extremamente flexíveis).”

Tem gente que argumenta que trabalhar em pares pode prejudicar a criatividade e impedir experimentações. Durante uma sessão em par, pode ser considerado rude desperdiçar o tempo de seu par tentando uma abordagem experimental que pode talvez não dar em nada. Então, a decisão mais segura possível tende a prevalecer sempre, mesmo que não seja a melhor possível.

Finalmente, a programação em par não fornece um dos benefícios chave da revisão de código posterior: ter uma pessoa com zero contexto analisando o código produzido. As duas pessoas do par compartilham contexto desde o início da sessão, o que não deve ser subestimado. As pessoas têm uma tendência a superestimar o valor de suas contribuições e criar laços emocionais com o que elas criam; por isso é tão importante conseguir uma outra pessoa que, livre desses laços, será capaz de oferecer um julgamento mais claro.

Não tem como fugir do fato de que programação em par, apesar de seus benefícios, requer um número par de pessoas, trabalhando ao mesmo tempo. Se seu time consiste de pessoas vivendo em diversos fusos horários diferentes (ou mesmo uma equipe local mas com horário flexível), é fácil decidir: revisão de código na cabeça. Se a equipe não se encaixa na descrição acima, então faça uma tentativa com programação em par. Contanto que você se esforce para acomodar e ter empatia com diferentes tipos de personalidades e não torne obrigatório, programação em par pode ser benéfica para sua equipe.

Finalmente, não há nada lhe impedindo de usar ambas práticas. Você pode adotar programação em par como o *Modus Operandi* padrão e deixar a revisão de código para preencher os vácuos onde o pareamento não funciona tão bem. De um jeito ou de outro, abraça a automação. Imagine que você escreve para uma publicação, como uma revista. Após finalizar um rascunho, ele precisa ser submetido para revisão. Será que faz sentido para a editora gastar todo seu tempo procurando por erros de ortografia? É claro que não! Nós temos ferramentas para checar tais erros de forma automática, liberando a editora para procurar por problemas em um nível mais alto, como vocabulário mal escolhido, falta de coesão, tom inapropriado, etc..

Com software não é diferente. Ao usar uma ferramenta de revisão de códigos automatizados, você pode eliminar muitas das discussões inúteis que acontecem frequentemente em revisões de código ou sessões de programação em par. Não vai ter mais briga sobre convenções estéticas e de nomenclatura, posição de chaves e outras trivialidades inúteis. Você também pode usar uma ferramenta de análise estática para lhe alertar sobre potenciais bugs e oportunidades para refatoração. Assim, o revisor/navegador fica livre para focar em preocupações de mais alto nível que necessitam da criatividade, inteligência e empatia

humanas.

Considerações Finais

Neste estudo entendemos os padrões estruturais e como os mesmos podem ser aplicados aos projetos de software orientados a objetos. Esses padrões são fundamentais para a estabilidade do software e facilitam o teste e manutenção ao longo do ciclo de vida do produto.

UNIDADE IV – Requisitos de Qualidade de Software

Objetivos de aprendizagem

- Considerações práticas
- Requisitos de qualidade de software
- Caracterização de defeitos

Um ponto interessante para adotar questões de qualidade é definir quais são os requisitos de qualidade para cada elemento do software. Neste estudo, vamos abordar esses elementos e entender como cada um deles pode ser utilizado para obter software de qualidade.

Considerações práticas

Para se garantir a qualidade de um produto durante seu desenvolvimento existe um conjunto de métodos e técnicas que devem ser implementadas durante todo o processo (Sommerville, 2011). Neste estudo, veremos a definição de algumas dessas atividades. Conhecer os conceitos de erro, defeito e falha é o requisito básico para se construir um processo de garantia de qualidade. É preciso saber identificar o problema, antes de investigar sua proveniência e, desta forma, saber corrigi-lo.

- Erro: trata-se de um engano de um indivíduo
- Defeito: deficiência mecânica ou algorítmica que se ativada pode levar a uma falha
- Falha: evento notável onde o sistema viola suas especificações

Quanto antes a presença do defeito for revelada, menor o custo de correção do defeito e maior a probabilidade de corrigi-lo corretamente. A solução é introduzir atividades de VV&T ao longo de todo o ciclo de desenvolvimento.

Tabela 2. Tipos de Defeitos (Sommerville, 2011)

Informação Estranha	Informação desnecessária
Fato Incorreto	Informação que não é verdadeira para as condições especificadas
Inconsistência	Informações conflitantes

Ambiguidade	Informação passível de ter múltiplas interpretações
Omissão	Informação necessária não incluída

As atividades de VV&T atuam na garantia de qualidade, assegurando que o software cumpra as especificações definidas e atenda às necessidades de seus usuários. Definições:

- Validação: avalia um sistema ou componente para determinar se ele satisfaz os requisitos para ele especificados. “Estamos construindo o produto certo?”. O software deve atender às necessidades dos usuários
- Verificação: avalia um sistema ou componente para determinar se os produtos de uma dada atividade de desenvolvimento satisfazem as condições impostas no início desta atividade. “Estamos construindo certo o produto?” Os artefatos construídos devem estar de acordo com a especificação do software

Os principais métodos de Validação e Verificação são:

- Estática: Revisões de Software (Revisões aos Pares, Inspeções de Software, entre outros). Denomina-se estática, pois não envolve a execução do produto, mas apenas leitura de artefatos, como código
- Dinâmica: Testes de Software (Testes de unidade, de integração, do software, entre outros)

São processos ou atividades para leitura de um artefato de software visando assegurar que este cumpra sua especificação e atenda às necessidades de seus usuários. Tem por objetivo realizar validação e verificação estática de artefatos de software. Pode ser aplicada a qualquer artefato produzido ao longo do processo de desenvolvimento de software.

#REFLITA#

Procure identificar sempre a causa dos defeitos. Trabalhar no sintoma geralmente é mais fácil mas tende a se repetir no longo prazo. Além disso, quando um erro torna-se recorrente a manutenção do mesmo pode introduzir novos erros entrando em um ciclo contínuo de falhas.

#REFLITA#

Requisitos de qualidade de software

O projeto de software é uma etapa fundamental no desenvolvimento de sistemas de grande porte e complexidade. Dentro deste contexto, a arquitetura de software é fundamental para o desenvolvimento de linhas de produção de software onde se tem um conjunto de funcionalidades concebidas e implementadas a partir da mesma arquitetura base. Entretanto, antecedendo a etapa de projeto da arquitetura de software, há a necessidade de fazer o levantamento dos requisitos do sistema (Sommerville, 2011).

De um modo geral, o conjunto de requisitos de um sistema é definido durante as fases iniciais do processo de desenvolvimento. Tal conjunto de requisitos é visto como uma especificação do que deveria ser implementado. Os requisitos são descrições de como o sistema deveria se comportar, e contêm informações do domínio da aplicação e restrições sobre a operação do sistema.

Durante a fase de eliciação de requisitos, um projetista ou arquiteto de software faz uso de sua experiência a fim levantar os requisitos, buscando identificar características do sistema a ser desenvolvido. Além disso, informações do domínio juntamente com informações de estilos arquiteturais existentes podem ser utilizados como fontes de dados que auxiliam na identificação dos requisitos.

Outro recurso que pode ser usado pelo projetista é construir cenários. Os cenários de uso oferecem suporte a requisitos específicos e visam tanto a eliciação quanto à análise de requisitos. Uma vez que o conjunto de requisitos tenha sido obtido, o projetista/arquiteto de software estará em condições de iniciar o projeto da arquitetura de software, conforme ilustrado na Figura 6.

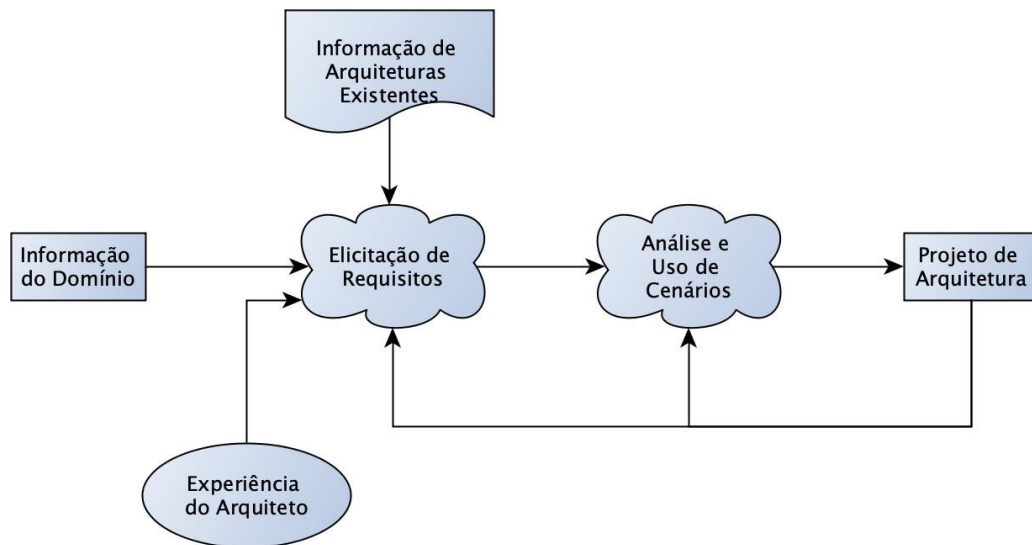


Figura 6. Elicitação de requisitos (Sommerville, 2011)

Este processo de levantamento e análise de requisitos, em conjunto com o uso de cenários, é usado no suporte da definição da arquitetura de software, como é discutido ao longo do artigo. É importante observar que a etapa de projeto arquitetural pode precisar fazer uso de cenários de uso ou mesmo uma reanálise a fim de refinar a arquitetura a ser empregada no sistema a ser desenvolvido.

O processo de desenvolvimento baseado na arquitetura considera a arquitetura de software como fator orientador do processo. Isto implica em colocarmos os requisitos não funcionais associados à arquitetura como principais aspectos do processo de desenvolvimento. Note que o desenvolvimento de um sistema de software centrado na arquitetura inicia-se com um arquiteto de software, de posse de um conjunto de requisitos do sistema. Nesse momento, busca-se identificar qual estilo ou combinação destes oferece suporte mais adequado a esses requisitos e, portanto, derivar uma arquitetura de software que atenda às características do sistema a ser desenvolvido. Vale ressaltar que a complexidade de um sistema de software é determinada tanto por seus requisitos funcionais – o que ele faz – quanto requisitos não funcionais – como ele faz. Um requisito de sistema de software que especifica uma função que o sistema ou componente deve ser capaz de realizar. Estes são requisitos de software que definem o comportamento do sistema, ou seja, o processo ou transformação que componentes de software ou hardware efetuam sobre as entradas para gerar as saídas. Esses requisitos capturam as funcionalidade sob o ponto de vista do usuário.

Adicionalmente, em engenharia de sistemas de software, um requisito não funcional de software é aquele que descreve não o que o sistema fará, mas como ele fará. Assim, por exemplo, têm-se requisitos de desempenho, requisitos da interface externa do sistema, restrições de projeto e atributos da qualidade. A avaliação dos requisitos não funcionais é feita, em parte, por meio de testes, enquanto que outra parte é avaliada de maneira subjetiva. Perceba que tanto os requisitos funcionais quanto os não funcionais possuem importância no desenvolvimento de um sistema de software. Entretanto, os requisitos não funcionais, também denominados de atributos de qualidade, têm um papel relevante durante o desenvolvimento de um sistema, atuando como critérios na seleção e/ou composição de uma arquitetura de software, dentre as várias alternativas de projeto. Cabe salientar que à medida que os sistemas tornam-se maiores e mais complexos, o suporte a requisitos não funcionais depende cada vez mais de decisões tomadas no projeto da arquitetura de software. Trata-se de uma visão compartilhada pelos profissionais da área e, especificamente, pela comunidade de arquitetura de software.

Requisitos não funcionais são aqueles que não estão diretamente relacionados à funcionalidade de um sistema. O termo requisitos não funcionais é também chamado de atributos de qualidade (BOEHM, 2000). Os requisitos não funcionais têm um papel de suma importância durante o desenvolvimento de um sistema, podendo ser usados como critérios de seleção na escolha de alternativas de projeto, estilo arquitetural e forma de implementação. Desconsiderar ou não considerar adequadamente tais requisitos é dispendioso, pois torna difícil a correção uma vez que o sistema tenha sido implementado. Suponha, por exemplo, que uma decisão tenha sido feita de modularizar a arquitetura de um sistema de modo a facilitar sua manutenção e adição de novas funcionalidades. Entretanto, modularizar um sistema adicionando uma camada a mais pode comprometer um outro requisito, o de desempenho.

Portanto, faz-se necessário definir logo cedo quais requisitos não funcionais serão priorizados na definição de uma arquitetura.

Os requisitos não funcionais abordam aspectos de qualidade importantes em sistemas de software. Se tais requisitos não são levados em consideração, então o sistema de software poderá ser inconsistente e de baixa qualidade, conforme discutido acima. Para tanto, quanto mais cedo forem definidos os critérios arquiteturais, mais cedo o projetista pode identificar o estilo, ou combinação de estilos, mais apropriado ao sistema considerado. Ao desenvolver um novo sistema de software bem como sua arquitetura, os projetistas e engenheiros de software apresentam um conjunto de atributos de qualidade ou requisitos não funcionais que o sistema deveria suportar. Exemplo destes requisitos são desempenho, portabilidade, manutenibilidade e escalabilidade.

A arquitetura de software deveria oferecer suporte a tais requisitos. Isto resulta da associação existente entre arquitetura de software e requisitos não funcionais. Importante observar que cada estilo arquitetural suporta requisitos não funcionais específicos. A estruturação de um sistema é determinante no suporte oferecido a um requisito não funcional. Por exemplo, o uso de camadas permite melhor separar as funcionalidades de um sistema, tornando-o mais modular e facilitando sua manutenção. Considere, por exemplo, o padrão IEEE-Std 830-1993 que lista um conjunto de 13 requisitos não funcionais a serem considerados no documento de especificação de requisitos de software. Este padrão inclui, dentre outros, requisitos de desempenho, confiabilidade, portabilidade e segurança.

Embora haja um conjunto de propostas, consideradas como complementares, concentramos nossas atenções num conjunto de requisitos diretamente associados a um sistema de software e, especificamente, à arquitetura de software. Este conjunto é baseado numa classificação apresentada por Sommerville, onde é feita a distinção entre requisitos externos, de produto, e de processo. A Figura 7 é uma adaptação da proposta de Sommerville, onde consideramos os requisitos de produto, associados à arquitetura de software, bem como adicionamos outros não presentes na proposta original de Sommerville. É importante observar que a Figura 7 mostra um subconjunto de requisitos não funcionais, denominados de requisitos de produtos os quais estão associados à arquitetura de um sistema de software. Note que a classificação apresentada em ainda considera os requisitos de processo e requisitos externos como sendo requisitos não funcionais, além dos requisitos de produtos. Vamos focar aqui nos requisitos de produtos.

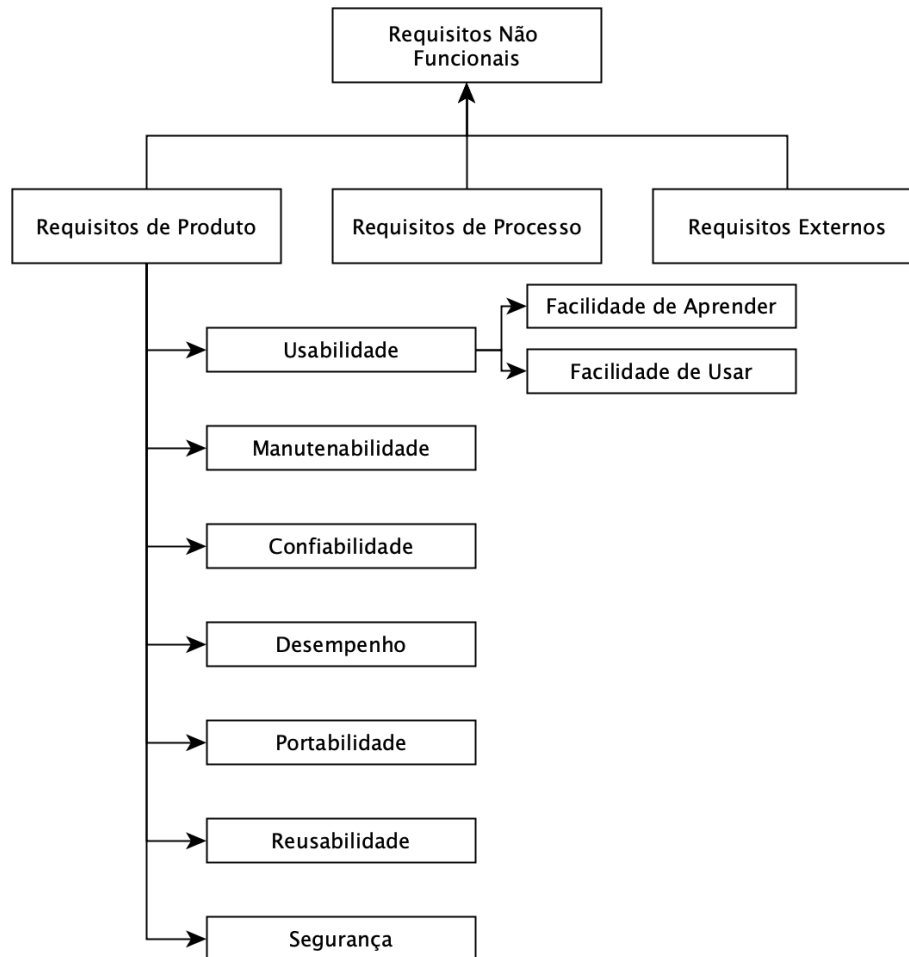


Figura 7. Tipos de requisitos não funcionais (Sommerville, 2011)

Usabilidade

Usabilidade é um dos atributos de qualidade ou requisitos não funcionais de qualquer sistema no qual ocorre interação entre o sistema e seres humanos. A noção de usabilidade vem do fato que qualquer sistema projetado para ser utilizado pelas pessoas deveria ser fácil de aprender e fácil de usar, tornando assim fácil e agradável a realização de qualquer tarefa. Requisitos de usabilidade especificam tanto o nível de desempenho quanto a satisfação do usuário no uso do sistema. Dessa forma, a usabilidade pode ser expressa em termos de:

- Facilidade de aprender: associado ao tempo e esforço mínimo exigido para alcançar um determinado nível de desempenho no uso do sistema.
- Facilidade de uso: relacionado à velocidade de execução de tarefas e à redução de erros no uso do sistema.

Os requisitos de usabilidade⁷ são coletados juntamente com outros requisitos usando algumas das técnicas de elicitación de requisitos como entrevistas ou observação. A coleta desses dados pode ocorrer, por exemplo, verificando o log de ações do usuário quando do uso de funcionalidade do sistema. Esses requisitos de usabilidade podem ser expressos por meio de métricas de usabilidade, expressas em termos de medidas de desempenho. Thayer apresentou um conjunto de critérios que podem ser utilizados durante a medição de usabilidade. A seleção de critérios a serem usados para mensurar a usabilidade depende do tipo de sistema. Exemplos de critérios de medição de usabilidade são apresentados na Figura 8.

1. Tempo para realizar um tarefa.
2. Percentual de tarefa concluído.
3. Percentual de tarefa concluído por unidade de tempo.
4. Taxa de sucessos/falhas.
5. Tempo consumido com erros.
6. Percentual de erros.
7. Número de comandos utilizados.
8. Número de comandos disponíveis não utilizados.
9. Frequência de uso de *ajuda* (help) ou documentação.
10. Número de vezes que o usuário expressa satisfação ou frustração.

Figura 8. Critérios de medição de usabilidade (Christensen e Thayer, 2002)

A definição das metas de usabilidade por meio de métricas é parte do processo denominado de engenharia de usabilidade. Neste processo, faz-se necessário também estabelecer os níveis desejados de usabilidade. Se, por exemplo, o usuário tem dificuldade em encontrar a funcionalidade desejada no sistema e, conseqüentemente, precisa recorrer à ajuda ou expressa insatisfação, então se observa que dois dos critérios da Figura 8 são considerados. A quantidade de vezes que essas ocorrências são observadas serve de indicador do suporte oferecido à usabilidade pelo sistema.

A usabilidade é um dos atributos de qualidade de um sistema e tem sido cada vez mais levada em consideração durante o desenvolvimento de software. A usabilidade pode ser afetada pelos componentes funcional (ou de aplicação) e de apresentação de um sistema. Mesmo que esses componentes sejam bem projetados, ainda assim a usabilidade poderá ficar comprometida se a arquitetura do sistema não levar em consideração a facilidade de efetuar uma modificação.

É importante acrescentar que a simples separação arquitetural entre componente de aplicação e apresentação em sistemas interativos não é suficiente para assegurar a usabilidade do mesmo. Assim, para determinar se uma arquitetura de software satisfaz a aspectos de usabilidade, torna-se necessário desenvolver cenários de uso do sistema. Em tais cenários, busca-se assegurar que informações corretas estejam disponíveis ao usuário no momento

⁷ Existe uma ferramenta para testar a usabilidade do software. Pode ser acessada no endereço: <http://www.labiutil.inf.ufsc.br/ergolist/>

adequado, bem como encaminhar corretamente as instruções/comandos dos usuários a componentes apropriados do sistema. Note que a arquitetura de software do sistema tem um papel de suma importância visto que tais informações serão trocadas entre componentes do sistema e entre estes e o usuário, fluindo através de conectores da arquitetura.

Manutenabilidade

O termo manutenção de software é geralmente empregado quando nos referimos às modificações feitas após o sistema de software ter sido disponibilizado para uso. Na realidade, o termo manutenabilidade é um tanto abrangente já que ele envolve tanto a atividade de reparo (de algum defeito existente no sistema de software) quanto a atividade de alteração/evolução de características existentes ou adição de novas funcionalidades não previstas ou capturadas no projeto inicial.

O reparo de um sistema de software ocorre quando defeitos são detectados, fazendo-se necessária a correção deles. A capacidade de efetuar um reparo depende do número de componentes do sistema. Por exemplo, se o sistema é monolítico, ou seja, constituído de um único componente, então tornar-se mais difícil efetuar o reparo se este sistema de software é de grande porte. No entanto, se o sistema de software é modularizado, então tende a ser mais fácil analisar e reparar o existente. Podemos dizer que a modularidade favorece a capacidade de fazer reparo, permitindo que defeitos fiquem confinados a poucos módulos, considerando-se que se tenha a funcionalidade adequadamente separada. Uma observação importante é que a necessidade de fazer reparo é minimizada à medida que a confiabilidade do sistema aumenta. Similarmente a outros sistemas, os sistemas de software evoluem ao longo do tempo, seja com a adição de novas funcionalidades ou com a modificação das existentes. Esta capacidade de evolução de um sistema de software é também influenciada pela modularidade do sistema. Note que se a arquitetura do sistema não considerar sua possibilidade de evolução por meio da adição e/ou alteração de funcionalidades do sistema, modificações tornar-se-ão cada vez mais difíceis à medida que o software evolui.

De um modo geral, a manutenabilidade é um dos requisitos mais relacionados com a arquitetura de um sistema de software. A facilidade de fazer alteração no sistema existente, seja adicionando ou modificando alguma funcionalidade, depende muito da arquitetura deste. Se considerarmos a necessidade de incrementar a quantidade de componentes encarregados do processamento de uma ligação telefônica ou de transações eletrônicas, então esta adição de novos componentes deve ocorrer sem a necessidade de modificar a arquitetura existente e ainda comprometer pouco ou nada o desempenho atual do sistema. Tal suporte à manutenabilidade (seja para correção ou evolução do sistema) deve ser facilmente acomodada pela arquitetura de software.

É importante lembrar que uma arquitetura de software define os componentes e conexões entre estes e, portanto, também definem sob que circunstâncias componentes ou conectores podem ser alterados. Dessa forma, uma arquitetura ou estilo arquitetural de um sistema de software deveria efetivamente acomodar as modificações que precisarem ser feitas

tanto durante seu desenvolvimento quanto após o sistema entrar em operação.

Confiabilidade

Confiabilidade de software é a probabilidade de o software não causar uma falha num sistema durante um determinado período de tempo sob condições especificadas. A probabilidade é uma função da existência de defeitos no software. Assim, os estímulos recebidos por um sistema determinam a existência ou não de algum defeito. Em outras palavras, a confiabilidade de software, geralmente definida em termos de comportamento estatístico, é a probabilidade de que o software irá operar como desejado num intervalo de tempo conhecido. Também, a confiabilidade caracteriza-se um atributo de qualidade de software o qual implica que um sistema executará suas funções como esperado.

Os requisitos de confiabilidade compreendem restrições sobre o comportamento do sistema de software em tempo de execução. Na realidade, tem-se um conjunto de métricas de confiabilidade de software associadas a esses requisitos. Geralmente, as falhas de um componente de software são de natureza transitória, ou seja, elas ocorrem apenas para algumas entradas (estímulos) enquanto o sistema poderá continuar operando normalmente em outras circunstâncias. Isto distingue o software do hardware já que as falhas no segundo são de natureza permanente. Vale ressaltar que falha é o que se observa pelos usuários (externamente) enquanto que os defeitos, de origem interna ao sistema, são os motivadores das falhas.

Não é tão simples relacionar a disponibilidade de um sistema de software a uma falha existente, pois isto depende de vários fatores, tais como grau de corrupção de dados em decorrência de algum defeito de software e tempo de reinicialização, dentre outros. Exemplos de métricas utilizadas para avaliar a confiabilidade de software compreendem:

- Disponibilidade: esta é uma medida de quão disponível o sistema estaria para uso, isto é, quão disponível o sistema estaria para efetuar um serviço solicitado por algum usuário. Por exemplo, um serviço de um sistema de software terá uma disponibilidade de 999/1.000. Isto significa que dentre um conjunto de 1.000 solicitações de serviço, 999 deverão ser atendidas. Esta métrica é muito importante em sistemas de telecomunicações, por exemplo.
- Taxa de ocorrência de falha: esta é uma medida da frequência na qual o sistema falha em prover um serviço como esperado pelo usuário, ou seja, a frequência na qual um comportamento inesperado é provável de ser observado. Por exemplo, se temos uma taxa de ocorrência de falha de 2/1.000, isto significa que 2 falhas são prováveis de acontecerem para cada 1.000 unidades de tempo.
- Probabilidade de falha durante fase operacional: esta é uma medida da probabilidade que o sistema irá comportar-se de maneira inesperada quando em operação. Esta métrica é de suma importância em sistemas críticos que requerem uma operação

contínua.

- Tempo médio até a ocorrência de falha ou *Mean Time To Failure* (MTTF): esta é uma medida do tempo entre falhas observadas. Note que esta métrica oferece um indicativo de quanto tempo o sistema permanecerá operacional antes que uma falha aconteça.

Qualquer métrica que venha a ser utilizada para avaliar a confiabilidade de um sistema dependerá da forma como o sistema é usado. Note ainda que o tempo é um fator considerado nas métricas. Ele é escolhido de acordo com a aplicação. Há sistemas de software que operam de forma contínua, enquanto outros operam de maneira periódica. Por exemplo, considere um caixa eletrônico de banco. Este é um exemplo de sistema que opera periodicamente, isto é, um caixa eletrônico encontra-se parte do tempo em operação, enquanto no restante do tempo fica ocioso. No exemplo de um caixa eletrônico de banco, uma unidade de tempo mais adequada é o número de transações. Assim, um exemplo de falha seria a perda de dados entrados por um usuário. Neste caso, a especificação de confiabilidade poderia ser a ocorrência de uma falha dessa natureza a cada 10.000 transações.

A arquitetura de software influenciará na confiabilidade de um sistema. O tempo médio até a ocorrência de uma falha ou MTTF poderá ser reduzido se houver replicação de componentes críticos. A perda de um desses componentes incorre em falha. Uma forma de evitar isto ou contornar a perda de um componente é provendo uma arquitetura tolerante a falha onde uma réplica de um componente assume o processamento do componente com falha, evitando assim qualquer interrupção na operação de um sistema. Outra alternativa é degradar o desempenho do sistema sobrecarregando um componente com um número maior de requisições do que ele foi projetado. Dessa forma, a qualidade do sistema é degradada, mas ainda assim o sistema continua a funcionar (embora de modo precário até que uma ação corretiva seja tomada). A medida de confiabilidade pode ser definida em termos do tempo médio entre a ocorrência de falhas, ou *Mean Time Between Failure* (MTBF). Esta medida é dada por:

$$MTBF = MTTF + MTTR^8$$

A medida de disponibilidade também pode ser descrita em termo do MTTF e é definida como:

$$\text{Disponibilidade} = MTTF \times 100\% / (MTTF + MTTR)$$

Se considerarmos essas medidas, é importante observar que se o MTTR é reduzido, então a disponibilidade e confiabilidade do sistema serão maiores. Isto pode ser obtido

⁸ MTTR ou Mean Time To Repair é o tempo médio de reparo

arquiteturalmente se a separação de interesses for considerada durante o projeto. Perceba que quão menor é o tempo para proceder o reparo da falha, mais rapidamente o sistema voltará a ficar operacional e, portanto, a ficar disponível. Este atributo de projeto leva a uma maior integração, bem como maior facilidade nas modificações necessárias no sistema. Note que adicionar componentes redundantes a um sistema de software implicará numa maior confiabilidade. Esta redundância é acrescentada na forma de verificações adicionais realizadas a fim de detectar erros antes que eles ocasionam falhas no sistema. Todavia, o uso de componentes redundantes acarreta numa redução de desempenho do sistema como discutido a seguir.

Desempenho

Desempenho é um atributo de qualidade importante para sistemas de software. Considere um sistema de uma administradora de cartões de crédito. Em tal sistema, um engenheiro de software poderia considerar os requisitos de desempenho para obter uma resposta de tempo para autorização de compras por cartão. Note que os requisitos de desempenho têm impacto mais global sobre o sistema e, por essa razão, estão entre os requisitos não funcionais mais importantes. Contudo, é geralmente difícil lidar com os requisitos de desempenho e com outros requisitos não funcionais uma vez que eles estão em conflito, conforme discutido acima. No início da atividade de projeto da arquitetura de software torna-se necessário definir quais requisitos não funcionais serão priorizados, dada a possibilidade de conflito entre eles.

Adicionalmente, desempenho é importante porque afeta a usabilidade de um sistema. Se um sistema de software é lento, ele certamente reduz a produtividade de seus usuários ao ponto de não atender às suas necessidades. Também, se o sistema de software requer muito espaço em disco para armazenamento de informações, pode ser oneroso utilizá-lo. Por exemplo, se um sistema de software exige muita memória para ser executado, ele pode afetar outras aplicações que são executadas no mesmo ambiente. Além disso, ele pode executar tão lentamente que o sistema operacional tenta balancear o uso de memória entre as diversas aplicações. De um modo geral, o requisito de desempenho pode ser decomposto em termos de tempo e espaço, como ilustrado na Figura 9.

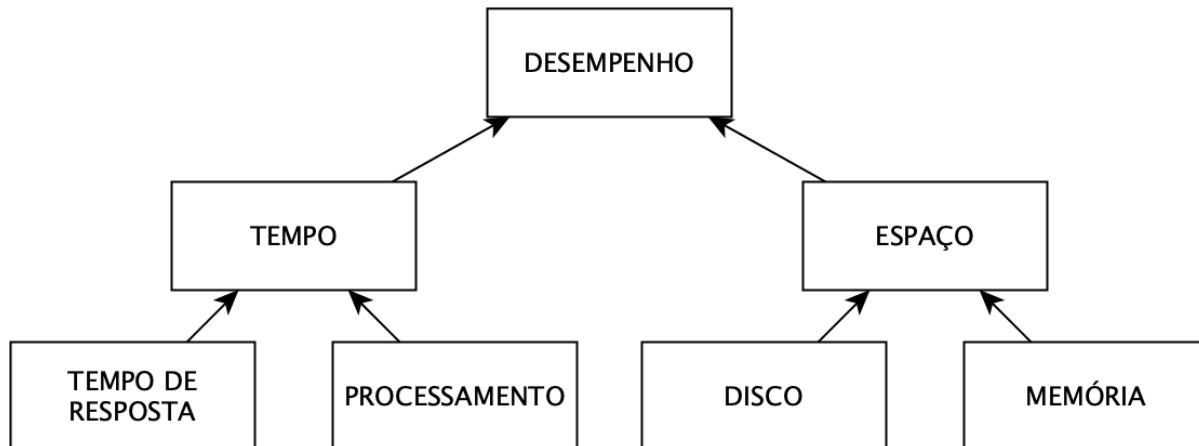


Figura 9. Fatores de desempenho (Sommerville, 2011)

O requisito de desempenho restringe a velocidade de operação de um sistema de software. Especificam o tempo de resposta de um sistema de software aceitável para usuários. Neste caso, um projetista poderia especificar que o sistema deveria responder à solicitação de um serviço específico de um usuário dentro de um intervalo de 2 segundos. Por exemplo, num caixa eletrônico, após o usuário inserir o cartão magnético do banco no local apropriado (leitor do equipamento), o sistema deveria exibir uma nova tela, num intervalo de 2 segundos, requerendo que o usuário digite sua senha de conta corrente. Numa outra situação, o usuário pode ser solicitado a digitar sua senha e não o faz dentro de um período de 20 segundos, quando então um *timeout* ocorre e o sistema retorna à tela inicial.

Estes requisitos especificam a quantidade de dados que deveria ser processada num determinado período de tempo. Um exemplo seria exigir que o sistema de software possa processar, no mínimo, 80 transações por segundo. Os requisitos de temporização especificam quão rapidamente o sistema deveria coletar dados de entrada de sensores antes que outras leituras de dados de entrada, feitas posteriormente, sobrescrevem os dados anteriores. Assim, por exemplo, poderia ser especificado que o sistema deveria efetuar leitura de dados 500 vezes por segundo, como condição mínima. Em alguns casos, os requisitos de espaço podem ser considerados. Aqui, podemos nos referir à memória principal ou secundária. Por exemplo, a memória principal para executar uma aplicação poderia ser considerada como um requisito de desempenho uma vez que ela está relacionada ao comportamento do sistema em tempo de execução.

É importante observar que o desempenho depende da interação existente entre os componentes de um sistema de software e, portanto, está muito associado à arquitetura. Neste caso, os mecanismos de comunicação utilizados pelos componentes de um sistema têm influência sobre o desempenho obtido. Conforme vimos anteriormente, desempenho está relacionado a outros requisitos não funcionais. Por exemplo, a confiabilidade melhora com o uso de componentes redundantes. Todavia, o desempenho fica bastante comprometido, implicando na sua redução.

Portabilidade

A portabilidade⁹ pode ser definida como a facilidade na qual o software pode ser transferido de um sistema computacional ou ambiente para outro. Em outras palavras, o software é dito portátil se ele pode ser executado em ambientes distintos. Note que o termo ambiente pode referir-se tanto à plataforma de hardware quanto a um ambiente de software como, por exemplo, um sistema operacional específico. De um modo geral, a portabilidade refere-se à habilidade de executar um sistema em diferentes plataformas. É importante observar que à medida que aumenta a razão de custos entre software e hardware, a portabilidade torna-se cada vez mais importante. Adicionalmente, podemos ter a portabilidade de componentes e a portabilidade de sistemas. Esta última situação pode ser vista como caso especial de reusabilidade. O reuso de software ocorre quando todo o sistema de software é utilizado, implementando-o em diferentes sistemas computacionais. A portabilidade de um componente ou sistema de software é proporcional à quantidade de esforço necessário para que funcione num novo ambiente. Se uma quantidade menor de esforço é exigida quando comparada ao trabalho de desenvolvimento, então o sistema é dito portátil. Dois aspectos relevantes na portabilidade de programas são a transferência e adaptação. A transferência é o movimento de componente (código de um programa e dados associados) de um ambiente para outro. A adaptação engloba as modificações exigidas para fazer com que o programa seja executado em um novo ambiente. Cabe salientar que o ambiente no qual um sistema de software opera é normalmente composto de hardware, sistema operacional, sistema de entrada e saída (E/S), bem como a aplicação. Uma abordagem geral que poderia ser adotada para obter um sistema portátil tentaria separar as partes do sistema que dependem do ambiente externo numa camada ou interface de portabilidade, conforme ilustrado na Figura 10.

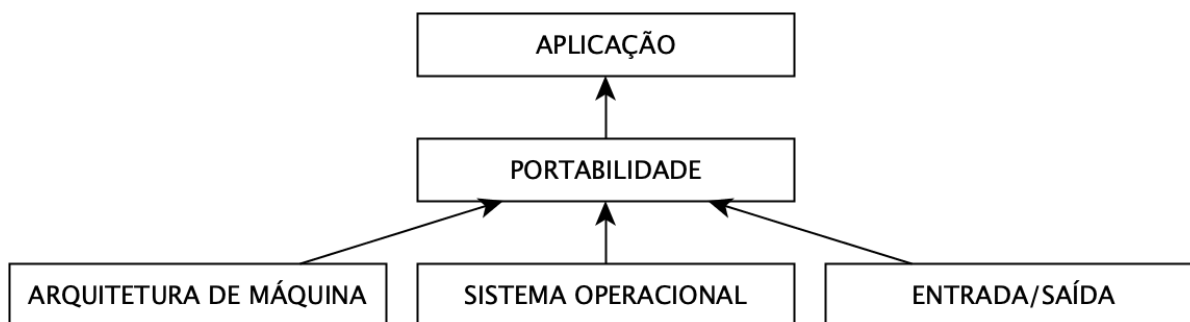


Figura 10. Separação de partes de um sistema computacional (Sommerville, 2011)

A interface de portabilidade mostrada na figura acima poderia ser vislumbrada e projetada como um conjunto de tipos de dados abstratos ou objetos, os quais iriam encapsular os elementos não portáveis procurando esconder características do software da aplicação. Assim, quando o sistema de software muda de hardware ou sistema operacional, apenas a

⁹ A tecnologia Java surgiu como uma promessa de portabilidade nos anos 90.

interface de portabilidade precisaria ser alterada. Alguns problemas de portabilidade surgem, geralmente, devido à adoção de diferentes convenções para representar informação em arquiteturas de máquinas distintas, ou seja, hardware diferente.

Reusabilidade

Uma característica das engenharias é fazer uso de projetos existentes a fim de reutilizar componentes já desenvolvidos, objetivando minimizar o esforço em novos projetos. Dessa forma, componentes que já tenham sido desenvolvidos e testados podem ser reutilizados. Considere os elevados níveis de reusabilidade que encontramos tanto na indústria de automóveis quanto de aparelhos eletrônicos. Na indústria de automóveis, por exemplo, um motor é geralmente reutilizado de um modelo de carro para outro. Em Engenharia de Software, à medida que aumenta a pressão para reduzir custos de desenvolvimento e manutenção de sistemas de software, bem como pela obtenção de sistemas com qualidade elevada, torna-se necessário considerar a reusabilidade como requisito não funcional no desenvolvimento de novos sistemas. O reuso pode ser visto sob diferentes perspectivas. Ele pode ser orientado a componentes, orientado a processos ou orientado ao conhecimento específico de um domínio. Note que ainda poderíamos considerar o reuso de requisitos. Entretanto, aqui, iremos concentrar nossa atenção no reuso orientado a componentes. Exemplos desse tipo de reuso são:

- Aplicação: toda a aplicação poderia ser reutilizada
- Subsistemas: os principais subsistemas de uma aplicação poderiam ser reutilizados
- Objetos ou módulos: componentes de um sistema, englobando um conjunto de funções, podem ser reutilizados
- Funções: componentes de software que implementam uma única função podem ser reutilizados

Dois tipos de reuso que estamos mais interessados são o reuso de subsistemas e objetos ou componentes, que chamaremos, simplesmente, de reuso de componentes. Este tipo de reuso não envolve apenas o código, mas também abrange a arquitetura e projeto associados. É importante observar que podemos obter ganhos se reutilizarmos tanto projetos quanto arquiteturas. Isto diminui esforços de desenvolvimento e necessita de menos alterações ou adaptações. Na realidade, quando se tem em mente que é necessário prover suporte à fácil alteração indiretamente, resultando componentes reutilizáveis. Dessa forma, o requisito reusabilidade pode envolver a arquitetura de um sistema de software ou componentes deste. O que definirá quão fácil será alcançar componentes reutilizáveis é a interdependência ou acoplamento entre os componentes. Por exemplo, uma biblioteca de componentes que oferece um conjunto de funcionalidades já implementadas e testadas disponibiliza ao usuário (programador) um recurso valioso, já que basta incluir esses componentes ao seu código e acessar suas funcionalidades por meio de sua interface.

Segurança

Em um sistema de software, este requisito não funcional define a segurança de que acessos não autorizados ao sistema e dados associados não serão autorizados. Portanto, é garantida a integridade do sistema quanto a ataques intencionais ou acidentes. Dessa forma, a segurança é vista como a chance de que a ameaça de algum tipo será repelida. Complementarmente à medida que os sistemas de software tornam-se distribuídos e conectados a redes externas, os requisitos de segurança vão se tornando cada vez mais importantes. Exemplos de requisitos de segurança são:

- Apenas pessoas que tenham sido autenticadas por um componente de controle acesso e autenticação poderão visualizar informações dado que a confidencialidade permite esse tipo de acesso apenas às pessoas autorizadas
- As permissões de acesso ao sistema podem ser alteradas apenas pelo administrador de sistemas
- Deve ser feito cópias (*backup*) de todos os dados do sistema a cada 24 horas e estas cópias devem ser guardadas em um local seguro, sendo preferencialmente num local diferente de onde se encontra o sistema
- Todas as comunicações externas entre o servidor de dados do sistema e clientes devem ser criptografadas

Requisitos de segurança são essenciais em sistemas críticos, tais como sistemas de controle de voo de aeronaves, uma vez que é impossível confiar num sistema deste tipo se ele não for seguro. Assim, pode-se dizer que todos os sistemas críticos possuem associados a eles requisitos de segurança. Os requisitos não funcionais de segurança envolvem diferentes aspectos. A Figura 11 mostra tipos de segurança que podemos encontrar.

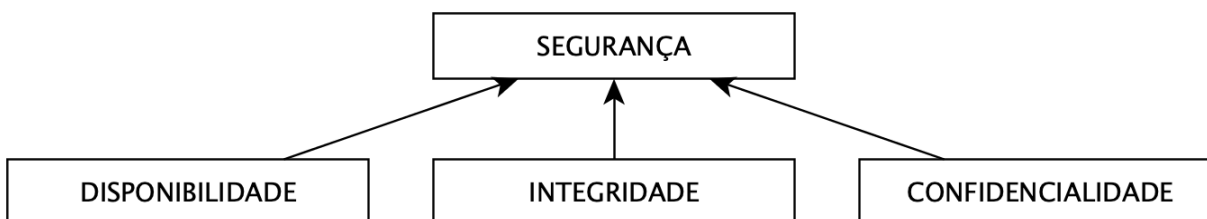


Figura 11. Tipos de segurança (Sommerville, 2011)

Também é importante considerar as diferentes ênfases encontradas para segurança:

- Disponibilidade: Refere-se a assegurar o sistema contra qualquer interrupção de

serviço

- Integridade: O foco na integridade ocorre principalmente em sistemas comerciais, onde se busca assegurar que acesso ou atualizações não autorizadas ocorram
- Confidencialidade: A ênfase aqui é a de não permitir a revelação não autorizada de informações

Note que para satisfazer ao requisito de qualidade não funcional de segurança em um sistema de software, alguns métodos podem ser empregados. Esses métodos podem ser vistos como um refinamento da meta de prover segurança a um sistema de software. Como exemplos desses métodos, podemos considerar:

- Identificação: identifica o nome do usuário, informando ao sistema quem está utilizando-o.
- Autenticação: visa assegurar que os usuários são, de fato, quem afirmam ser. Para tanto, fazem um teste de identidade. Este método envolve alguns aspectos, tais como:
 - Tipo de protocolo usado: isto requer operação de senha.
 - Quantidade de autenticações: pode requerer uma única senha ou múltiplas senhas ou procedimentos. Por exemplo, alguns bancos já fazem uso de múltiplas senhas durante operação de autenticação.
 - Partes envolvidas: isto pode envolver a autenticação de uma parte envolvida (cliente) ou de ambas as partes envolvidas (cliente e sistema).
 - Tempo de acesso: busca limitar o tempo de acesso ao sistema a fim de reduzir qualquer tipo de ameaça.

Adicionalmente, outros elementos podem ser observados. Vamos ver alguns deles logo abaixo.

- Auditoria de segurança: objetiva habilitar pessoal autorizado a monitorar o sistema e, seletivamente, rastrear eventos importantes.
- Alarme: esta operação visa prevenir acessos, potencialmente suspeitos às informações vitais ou dados do sistema, notificando esses acessos à supervisão de segurança do sistema ou devidas autoridades.

A Figura 12 apresenta alguns métodos que podem ser empregados em requisitos de segurança.

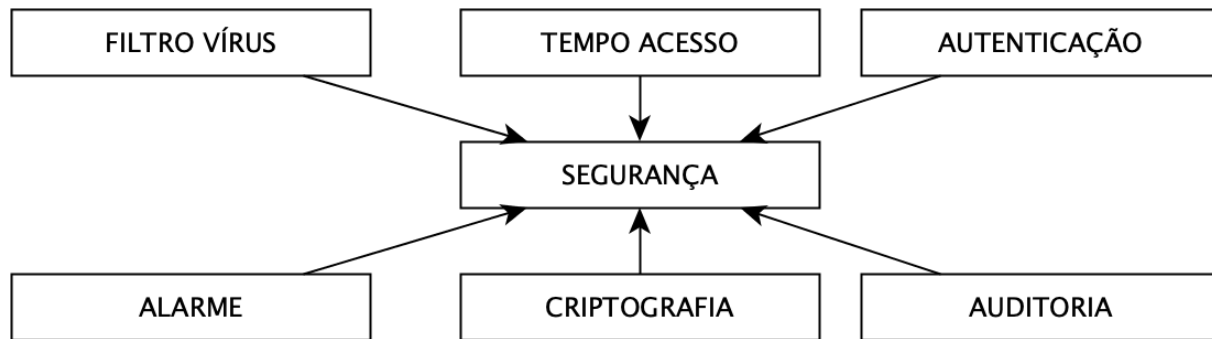


Figura 12. Métodos usados para prover segurança (Sommerville, 2011)

É importante observar que a arquitetura de um sistema de software deve levar em consideração esses aspectos a fim de atender aos requisitos de segurança. Entretanto, o tipo de sistema determinará quais fatores precisam ser levados em conta. Assim, poderá haver a inserção de componentes específicos de segurança bem como a conexão destes com outros componentes funcionais.

A elicitação de requisitos funcionais e não funcionais é etapa fundamental no desenvolvimento de sistemas de software. Elementos de entrada desse processo de elicitação compreendem os principais influenciadores do sistema, envolvendo projetista, arquiteto e usuários. Aliado a isso, a experiência do arquiteto de software é de grande importância. Como saída deste processo, tem-se um conjunto de requisitos funcionais oferecendo suporte às funcionalidades do sistema e uma lista de requisitos não funcionais oferecendo suporte à arquitetura de software. Cabe destacar que, quando da análise de arquiteturas candidatas para um sistema de software, um arquiteto ou engenheiro de software considera os requisitos não funcionais como um dos principais critérios para sua análise.

#REFLITA#

A segurança é um elemento de qualidade que geralmente é ignorada em projetos de software. Isso tem várias causas mas, em grande parte, a causa é cultural.

#REFLITA#

Caracterização de defeitos

Defeitos estão presente em projetos de software por anos e apesar de todo esforço investido para apoiar sua identificação e eliminação, continuam sendo um desafio para os engenheiros de software. De acordo com IEEE Std. 1044-2009, defeito é uma imperfeição ou deficiência em um produto de trabalho, onde esse produto de trabalho não cumpre suas exigências ou especificações e precisa ser reparado ou substituído. O combate aos defeitos de software é dificultado por fatores como complexidade implícita no desenvolvimento de

software, prazos curtos etc. Na indústria, quando adotado, este combate não ocorre de forma sistemática na maioria dos casos, o que dificulta seu entendimento. Para combater os defeitos de software de forma sistemática, criar ou adotar uma taxonomia de defeito é uma das atividades mais relevantes para o sucesso deste combate, taxonomia essa que será o pilar para classificação dos defeitos encontrados no processo de desenvolvimento, possibilitando um melhor entendimento sobre os principais defeitos presente nos projetos de software.

Para lidar com os defeitos de software é interessante categorizá-los via uma taxonomia. Taxonomia é uma ciência que estuda e define grupos com base em características comuns, dando nome para estes grupos. Originalmente, seu objetivo foi classificar organismos vivos na biologia, mas atualmente é aplicada de forma abrangente, principalmente pelas diversas ciências que fazem uso de taxonomias para classificar seus objetos de estudo. Cada grupo definido pela taxonomia recebe uma nota e eles podem ser agregados para formar supergrupos de maior pontuação, criando uma classificação hierárquica. Um exemplo do uso da taxonomia ocorre quando um cientista precisa classificar uma nova espécie de animal, para isto o cientista classifica este dentro de uma das categorias já existentes, baseado em uma lógica previamente estabelecida, verificando sua família e outras características para no final atribuir um nome adequado à nova espécie encontrada. O processo de criação de uma taxonomia exige profunda análise, seguida pela síntese do conhecimento adquirido e por fim uma definição, geralmente este processo envolve diversas revisões. Uma taxonomia deve ser sempre expansível, se adaptando ao surgimento de novas categorias, pois taxonomias em constante evolução tendem a ser mais úteis e atuais, pelo simples fato de que construir uma taxonomia perfeita na primeira tentativa é muito difícil.

Na engenharia de software, o uso de taxonomias é amplamente adotado. Um dos cenários que faz uso de taxonomia visa classificar defeitos encontrados no processo de desenvolvimento, citado como taxonomia de defeito. Taxonomia de defeito pode ser definido como um sistema hierárquico de categorias projetado para ajudar na classificação de faltas e falhas. Uma boa taxonomia de defeito é uma ferramenta útil para informar os iniciantes sobre os tipos de defeitos que podem ser encontrados. O principal fator que influencia na adoção ou criação de uma taxonomia de defeito são suas categorias de nível superior, que podem ser genéricas ou específicas para determinado domínio. O uso de taxonomia de defeito fornece informações sobre a distribuição de faltas e falhas em um projeto de software, sendo de grande valia para aprender sobre os tipos de defeitos que estão ocorrendo no processo de desenvolvimento.

O papel executado pelas taxonomias de defeitos está diretamente ligado ao combate sistemático dos defeitos, pois para conseguir criar uma estratégia eficaz de combate, precisamos, primeiro, entender quais tipos de defeitos estão afetando nossos projetos, classificá-los e posteriormente pensar qual melhor maneira de eliminá-los. Tendo conhecimento dos tipos de defeitos, podemos analisar a distribuição dos defeitos que ocorrem nos projetos, visando planejar uma ação corretiva, como um treinamento para o time técnico ou adicionar/modificar uma etapa formal de inspeção de artefatos. Basicamente na maioria dos cenários, criar ou adotar uma taxionomia é o passo inicial para trabalhar de forma sistemática.

Teste de software é a processo de verificar se o produto está de acordo com as

especificações determinadas e que funciona corretamente no qual foi projetado. O objetivo dos testes é de achar erros ou falhas para que a equipe de desenvolvimento possa corrigi-lo o mais rápido possível, pois quanto mais rápido descobrir um erro, mais barato ele fica. Por essas características a disciplina de teste é considerável "destrutiva" e não "construtiva". Para que possamos dar prosseguimento na disciplina de testes temos que conhecer alguns conceitos como Defeito, Erro e falhas (Sommerville, 2011).

- Defeito: qualquer imperfeição ou inconsistência no produto do software ou em seu processo, um defeito é também uma não conformidade. O Defeito faz parte do produto, é algo que está implementada no código de maneira errada.
- Erro: um resultado de um defeito ou uma falha, como um retorno esperado, que por causa de uma falha teve um valor diferente do que esperado.
- Falha: está mais ligada ao hardware, como uma rede inacessível, queda de energia. Uma falha pode ocorrer por causa de um erro, por exemplo, houve um retorno de um valor não esperado, como *null*, isso é um erro, e por causa desse *null* ocasionou uma falha no sistema.

Os defeitos são ocasionados por vários motivos como:

- Usuários especificam os requisitos errados
- Analistas interpretam erradamente os requisitos
- Especificações Funcionais e Técnicas elaboradas erroneamente
- Codificação errada
- Dados errados
- Correções erradas de defeitos
- Inconsistência nos dados

Podemos classificar os defeitos em alguns itens como:

- Tratamento de erros: Não antecipar-se às falhas ou não evitando travamentos ou comportamentos anormais no programa.
- Erros de cálculos: Falhas decorrentes de cálculos computacionais, tais como divisão por zero, estouro de campos, algoritmos errados.
- Erros de inicialização: Estados iniciais e finais de variáveis errados;
- Manipulação de dados: Dados tratados ou manipulados erradamente ou não tratados, como datas inválidas ou dados *null*.

- Hardware: Falta de tratamento de retorno dos códigos dos periféricos como verificar se a impressora está ligada, ao fazer uma comunicação com outra máquina verificar se a rede está ativa.
- Controle do código fonte: Não controlar adequadamente o código fonte do sistema.
- Documentação: Não manter a documentação atualizada
- Testes: Não ter uma política de teste, um planejamento e execução adequada, não definir o escopo

Sempre que um defeito for identificado devemos reportá-los a área responsável, como algumas informações básicas

- Categoria do defeito (validação, algoritmo, acessos)
- Severidade
- Prioridade
- Descrição do defeito ou do comportamento resultado ou a própria mensagem de erro
- Reproduzível
- Reincidente
- Dados usados no teste

Todo defeito tem uma consequência, que vai desde suave a catastrófica. Tais consequências devem ser medidas em termos humanos e não computacionais e ele enumera os graus de consequência possíveis dos defeitos. O motivo de utilizar termos humanos é que um *bit* errado no software pode causar qualquer um dos graus indicados a seguir (Pressman, 2011).

1. Suave: um erro de grafia ou alinhamento
2. Moderado: saídas são redundantes ou enganosas, tendo um impacto pequeno, mas mensurável, na performance
3. Irritante: o comportamento do software se torna desumanizante, como contas sem valor e nomes truncados
4. Perturbador: o software recusa a manipular transações corretas
5. Sério: o software perde rastro das transações, não apenas da transação mas até de que ela ocorreu
6. Muito sério: o software realiza a transação incorreta
7. Extremo: quando os problemas citados acima não estão limitados a um pequeno grupo
8. Intolerável: quando não há como recuperar os dados

9. Catastrófico: o software falha

O importante é ter uma forma de classificar os defeitos para que os mesmos possam ser tratados com a devida importância e pela equipe adequada.

#ATENÇÃO#

Embora a classificação sugerida acima ser útil é interessante adapta-la ao vocabulário da organização que você trabalha.

#ATENÇÃO#

Considerações Finais

Os desenvolvedores fazem os softwares, mas os clientes são os que vão usá-los. Por isso, há a necessidade urgente de sistematizar formas de evitar os custos elevados consequente dos defeitos dos softwares e dos erros não intencionais dos usuários. Isto só será possível se forem priorizadas algumas características de qualidade de software: usabilidade, confiabilidade, funcionalidade e manutenibilidade. Sendo esses, os requisitos essenciais do produto de software exigidos pelos compradores e atendidos pelos vendedores. Assim veremos resultados positivos tendo softwares de qualidade e mais úteis.

No Brasil ainda existem empresas de software que não utilizam técnicas para melhorar a qualidade de seus produtos, por isso aquelas empresas que desenvolvem software de qualidade são mais competitivas, o que favorece a abertura de mercados externos para os softwares brasileiros e beneficiando o mercado interno que acabará por exigir software melhores e a menor preço. Não resta a menor dúvida que ainda restam muitos esforços em melhoria da qualidade de software. Por isso, é fundamental ter seu foco não apenas no produto (fazer software melhor), mais principalmente no processo (fazer melhor o software) e no cliente (fazer software mais fácil de usar).

UNIDADE V – Aspectos Operacionais de Qualidade de Software

Objetivos de aprendizagem

- Técnicas de gerência de qualidade de software
- Medidas de qualidade de software
- Ferramentas de Qualidade de Software

Grande parte dos aspectos da qualidade software pode ser automatizado via ferramentas. Neste estudo, vamos abordar algumas técnicas, medidas e ferramentas de qualidade para utilizá-las com foco a produzir software de qualidade.

Técnicas de gerência de qualidade de software

É impossível alcançar a perfeição em um software, então é importante definir o nível de qualidade que é suficiente para o seu cliente em questão e quais os custos que estão associados. Todas as atividades no desenvolvimento de um software possui um custo, no controle de qualidade devem ser considerados três elementos: custo da falha, custo da prevenção e custo da avaliação. O custo de falha está associado aos defeitos entregue ao cliente, eles são divididos em falhas internas que inclui: retrabalho, análise de falhas e consertos de bugs e falhas externas que inclui: resolução de queixas, devolução do produto, suporte e trabalhos de segurança (Chrissis e Konrad, 2006).

Custo de prevenção pode ser considerado um investimento para que seja evitado erros na tentativa de realizar o trabalho certo de primeira vez. É um custo que acontece antes da criação do produto. É criado treinamento, realizado definição de métodos e procedimentos, planejamento da qualidade, testes de equipamentos e revisões técnicas formais. No custo de avaliação nos referimos a todos os gastos em procedimentos de verificação e em testes para a identificação dos erros do software após sua construção e antes que seja disponibilizado para uso. Podemos incluir nele manutenção dos equipamentos, tempo gasto na automação e inspeções.

O controle de qualidade envolve uma série de inspeções, revisões e teste com os propósitos de assegurar que todos os procedimentos e padrões sejam seguidos. Controle de qualidade referente a softwares procura identificar inconformidades dentro dos requisitos dados pelos usuários. O processo de qualidade tem seus próprios procedimentos a serem seguidos no desenvolvimento de um software, estes procedimentos devem ser fáceis de compreensão pelos engenheiros que estão desenvolvendo. Existem duas abordagens para o controle de qualidade, são elas: revisões de qualidade e avaliação automática de software.

Nas revisões de qualidade, a documentação e os processos utilizados são revisados por um grupo de pessoas, responsável por conferir se os padrões dos projetos foram seguidos e se a documentação está em conformidade com os padrões. O que não estiver conforme os padrões,

são anotados e submetidos à atenção da gerência do projeto. Já a avaliação automática de software envolve medição quantitativa de alguns atributos de software.

O foco do controle de qualidade é nas revisões e remoção dos erros antes mesmo da entrega final do produto. Um exemplo clássico de controle de qualidade são as inspeções de software feitas com base em critérios de entrada e saída bem definidos. O método de revisão envolve um grupo de pessoas que verificam parte ou todo o processo de desenvolvimento, com o objetivo de descobrir possíveis problemas. As conclusões da revisão são registradas formalmente e transferidas para o autor ou responsável sobre os problemas verificados. Existem três tipos de revisão:

- Tipo 1 – inspeções de projeto, onde é detectado os erros detalhados nos requisitos, nos projetos ou no código. É orientada por um checklist de erros
- Tipo 2 – revisão de progresso informa à gerência sobre o processo geral, se preocupa com os custos, planos e prazos
- Tipo 3 – revisões de qualidade, é realizada uma análise técnica da documentação do produto, com finalidade de encontrar inconsistências entre as especificações e o projeto, código ou documentação

O objetivo da equipe de revisões é detectar erros e mostrá-los ao projetista do documento. Elas são baseadas em documentos, porém não se limitam às especificações, projeto ou código. O benefício das revisões técnicas é a descoberta precoce dos defeitos de software, desta forma os defeitos podem ser corrigidos antes do próximo passo. As técnicas de revisão têm se mostrado até 75% efetivas na descoberta de falhas. No momento que é detectado o erro, o processo de revisão reduz os custos dos próximos passos na fase de desenvolvimento e na manutenção (Chrissis e Konrad, 2006).

Medidas de qualidade de software

A Engenharia de Software é uma disciplina que aplica os princípios de engenharia com o objetivo de produzir software de alta qualidade a baixo custo. Mas, o que vem a ser qualidade? Qualidade é o grau no qual um conjunto de características inerentes satisfaz aos requisitos. Do ponto de vista dos fornecedores de software, qualidade não é mais um fator de vantagem no mercado, mas uma condição necessária e pode-se dizer indispensável para que seja possível competir com sucesso. Para (BOEHM, 2000) *“um produto de software apresenta qualidade dependendo do grau de satisfação das necessidades dos clientes sob todos os aspectos do produto”*. As empresas que desenvolvem software passaram a valorizar não somente o seu produto, mas todas as fases que envolvem a sua elaboração e posterior entrega. Assim, a preocupação passou a ser com seu capital intelectual e o comprometimento com a qualidade, que é fator de competitividade e se mostra cada vez mais presente no plano estratégico das

organizações. Este novo cenário socioeconômico mundial, voltado à qualidade, tem forçado as organizações a reverem sua postura frente ao consumidor, ao empregado e à sociedade em geral.

Para conquistar clientes, diante da competição atual, é preciso encantá-los e seduzi-los. Qualidade intrínseca, preço acessível, flexibilidade, baixo custo de manutenção, valor de revenda, prazo de entrega, rapidez, variedade de opções, cordialidade no atendimento, boas condições de pagamento, imagem no mercado, segurança pessoal e ambiental são algumas entre as muitas possibilidades exploradas como vantagens competitivas. Diante do mencionado acima, pode-se concluir que a qualidade apresenta-se então como uma necessidade e não como um mero modismo. Qualidade pode ser tudo aquilo que alguém faz ao longo de um processo para garantir que um cliente, interno ou externo à organização, obtenha exatamente aquilo que deseja.

A qualidade de software não deve ser encarada como apenas um diferencial de mercado para que uma dada empresa de software possa conseguir vender e lucrar, mas sim como um pré-requisito que a empresa deve conquistar para conseguir colocar o produto no Mercado Global (BOEHM, 2000). Segundo a norma ISO/IEC 9126 (2000), *“qualidade é a totalidade das características de uma entidade que lhe confere a capacidade de satisfazer as necessidades explícitas e implícitas”*. Esta definição formal merece alguns complementos, principalmente para definir o que são entidades, às necessidades explícitas e as necessidades implícitas. A entidade é o produto do qual se está falando, que pode ser um bem ou um serviço. As necessidades explícitas são as próprias condições e objetivos propostos pelo produtor. As necessidades implícitas incluem as diferenças entre os usuários, a evolução no tempo, as implicações éticas, as questões de segurança e outras visões subjetivas.

Os fatos explícitos são externados pelo cliente. É o cliente quem define a qualidade, como qualidade dos projetos/processos, qualidade do produto final, manutenções corretivas, adaptativas e introdução de melhorias no produto. Os fatores implícitos dizem respeito aos fatores da qualidade do software, que são percebidos pelos desenvolvedores, que atendem aos fatores explícitos e principalmente à produção econômica do software. Só que para alcançar o topo da qualidade deve-se seguir algumas métricas que são necessárias para analisar qualidade e produtividade do processo de desenvolvimento e manutenção bem como do produto de software construído; e qualificar a performance técnica dos produtos do ponto de vista do desenvolvedor. Ainda na busca da qualidade existem alguns softwares que ajudam nessa empreitada e também algumas normas e padrões que são impostas onde permitirão a construção de um software que garanta a qualidade ao seu usuário final.

Embora a atual norma ISO/IEC 9126 (NBR 13596) enumere as características e sub-características um software, ela ainda não define como dar uma nota a um software em cada um destes itens. Se você não está familiarizado com o processo de avaliação de software, pode ter dificuldades em tentar utilizar a norma. Se você pretende avaliar um software segundo esta norma, deve tentar atribuir valores (como se fossem notas ou conceitos) a cada uma das sub-características. Segundo (Pressman, 2011) algumas características podem ser realmente medidas, como o tempo de execução de um programa, número de linhas de código, número de erros encontrados em uma sessão de teste ou o tempo médio entre falhas. Nestes casos, é

possível utilizar uma técnica, uma ferramenta ou um software para realizar medições. Em outros casos, a característica é tão subjetiva que não existe nenhuma forma óbvia de realizar a medição.

Importância de controlar os gastos com software. Analisar a performance dos resultados obtidos com o seu desenvolvimento e manutenção. Permitir uma padronização. Mas como medir o processo de software? Como medir, por exemplo, o tamanho das aplicações e a partir daí estimar prazos e custos realistas? Como medir a produtividade dos programadores e a partir daí avaliar o progresso no projeto e estimar os recursos necessários? Como medir a qualidade do software enquanto este está a ser desenvolvido e a partir daí determinar se são necessárias ações corretivas? Medidas são necessárias para:

- Analisar qualidade e produtividade do processo de desenvolvimento e manutenção bem como do produto de software construído.
- Qualificar a performance técnica dos produtos do ponto de vista do desenvolvedor.
- Medidas funcionais são necessárias para qualificar a performance dos produtos pela perspectiva do usuário.
- Devem ser independentes implementação. Das decisões do desenvolvimento técnico
- Utilizadas para comparar a produtividade de diferentes técnicas e tecnologias.

Possibilita realizar uma das atividades mais fundamentais do processo de gerenciamento de projetos que é o planejamento. A partir deste, identifica-se a quantidade de esforço, o custo e as atividades que serão necessárias para a realização do projeto. Há bem pouco tempo, ainda segundo (Sommerville, 2011), a única base para a realização de estimativas era a experiência da equipe técnica envolvida no projeto. Que pode ocasionar:

- Levava a atividades atropeladas ou não realizadas
- Custo de realização além do previsto
- Produtos com deficiência funcional
- Atraso na entrega do produto

O mercado está cheio de "ferramentas de produtividade". Com certeza o aumento de produtividade mais representativo será obtido quando conseguirmos estabelecer uma sistemática de métricas significativa para os resultados do desenvolvimento de software e efetivamente usá-la. A medição é algo comum no mundo da engenharia. Mas para engenharia de software está longe de ter uma medição padrão amplamente aceita e com resultados sem nenhum fator subjetivo. Tem-se dificuldade em concordar sobre o que medir e como avaliar o resultado das medições obtidas.

Medições e métricas auxiliam a entender o processo usado para se desenvolver um

produto. O processo é medido a fim de melhorá-lo e o produto é medido para aumentar sua qualidade. Alguns questionamentos sempre vêm à tona quando é feita uma tentativa de se medir alguma coisa que não tenha sido medida no passado. Isso é o que acontece com a engenharia de software (o processo) e com o software (o produto). As medidas são uma forma clara de avaliação de produtividade no desenvolvimento de software. Por meio da obtenção de medidas relativas à produtividade e à qualidade, é possível que metas de melhorias no processo de desenvolvimento sejam estabelecidas como forma de incrementar estes dois fatores. Métricas de software referem-se a uma ampla variedade de medidas de software. No contexto de gerenciamento de projeto de software, estamos preocupados com métricas de produtividade e de qualidade - medidas do resultado do desenvolvimento de software como uma função do esforço aplicado e medidas da "adequação ao uso" do resultado que é produzido. Algumas das razões para se medir um software são mencionadas pelo professor (Sommerville, 2011), tais como:

Tabela 3. Tipos de Medidas (Sommerville, 2011)

Medidas Diretas	Medidas Indiretas
• Custo • Esforço • Linhas de Código • Velocidade de Execução • Memória • Número de Erros	• Funcionalidade • Qualidade • Complexidade • Eficiência • Confiabilidade • Manutenibilidade

Formar uma base para as estimativas Ajudar na justificativa de aquisição de novas ferramentas ou de treinamentos adicionais. As métricas de software, do ponto de vista de medição, podem ser divididas em duas categorias: medidas diretas e indiretas. As medições de software podem ser organizadas em outras classes, as quais serão definidas a seguir conforme (Sommerville, 2011):

- Métricas da produtividade, baseadas na saída do processo de desenvolvimento do software com o objetivo de avaliar o próprio processo
- Métricas da qualidade, que permitem indicar o nível de resposta do software às exigências explícitas e implícitas do cliente

Métricas técnicas, nas quais encaixam-se aspectos como funcionalidade, modularidade, manutenibilidade, etc.

Sob outra ótica, é possível definir uma nova classificação das medições:

- Métricas orientadas ao tamanho, baseadas nas medições diretas da Engenharia de Software
- Métricas orientadas à função, que oferecem medidas indiretas
- Métricas orientadas às pessoas, as quais dão indicações sobre a forma como as pessoas desenvolvem os programas de computador

Embora o assunto de métricas de software seja extenso, é interessante estudá-las para entender como adequá-las ao seu processo de software. Essas métricas são úteis pois permite a comparação entre produtos, equipes, empresas e ao longo do tempo. Dentre os fatores que impedem desenvolvedores de conseguir estimar software de forma adequada:

- Requisito incompleto e muito subjetivo
- Falta de conhecimento de desenvolvedores sobre métricas de software para aplicar às suas próprias tarefas
- Gerentes de projeto querendo que usemos uma bola de cristal para tirar uma estimativa do nada sem dar tempo para fazermos uma análise dos requisitos
- Funcionalidade, projeto ou produto muito distinto do que já foi desenvolvido anteriormente, o que impede uma estimativa por analogia

Estimativas incorretas podem comprometer as entregas de diversas formas: acabam gerando um cronograma impreciso (muito apertado ou com muita folga que deixa a equipe ociosa), pode gerar mais custos para o projeto, horas extras para entregar no prazo acordado com o cliente, entre outros.

Outro ponto está relacionado com tempo e prazo. Podem parecer sinônimos, mas tempo e prazo de realização para uma atividade são conceitos distintos, embora complementares. Às vezes, até quem é responsável pela gestão de um projeto se confunde nesses conceitos no momento de solicitar uma estimativa. Na perspectiva de desenvolvimento de software, o tempo é o esforço total que você levaria para desenvolver um determinado requisito ou funcionalidade. Geralmente esse tempo é medido em horas, mas também pode ser medido em dias e, em casos raros, em semanas.

Exemplo: para fazer o front-end de um formulário de cadastro de clientes, você levaria 16 horas.

Já o prazo é um intervalo ou período de tempo dentro do qual alguma atividade deve ser realizada. Se você tem uma atividade que leva 8h para ser feita, não significa que ela tem o prazo de um dia ou que será feita em um dia. Mesmo porque, ninguém trabalha as 8 horas por dia.

Exemplo: (1) esse formulário de cadastro de clientes, que leva 16 horas, tem um prazo de cinco dias para ser desenvolvido e entregue; (2) você pode estimar que o formulário de cadastro, de 16 horas, será entregue em um prazo de três dias.

Ferramentas de Qualidade de Software

Praticamente, todos os conceitos estudados até então podem ser automatizados por ferramentas. Ou seja, não existe a necessidade de executá-los de maneira manual. Uma dica nesse caso é:

#ATENÇÃO#

Sempre que possível, automatize os elementos de qualidade de software em seu processo. Isso gera muitos benefícios e reduz o custo da qualidade.

#ATENÇÃO#

Podemos separar as ferramentas de qualidade segundo alguns grupos. Esse grupos, com algumas ferramenta que os representam, podem ser vistos na Tabela 4 abaixo.

Tabela 4. Tipos de Ferramentas

Tipo	Ferramenta	Exemplo	Finalidade
Qualidade de código	Análise estática Padronização Complexidade	Sonar PMD Metrics	Verificar a qualidade do código produzido e garantir que os padrões mínimos foram atendidos
Testes	Unidade Funcional Integração Performance Carga	JUnit Selenide Arquillian JMeter JMeter	Permitir a execução de testes de maneira automatizada, garantindo a facilidade na manutenção do software e publicação
Qualidade de análise	Simulador de regras Ferramenta case	DMN Simulator Enterprise Architect	Permitir que os artefatos de análise sejam produzidos com um nível de qualidade aceitável
Qualidade de operação	Inteligência operacional	Splunk	Coleta e indexa os logs de operação que são produzidos pelo software no ambiente de produção
Qualidade de atendimento	Requisição de serviço	ZenDesk Bugzilla	Permite o controle de requisições que partes do usuário e são gerenciadas durante o ciclo de

	Bugs Mudanças	Jira	vida do software
--	------------------	------	------------------

Essas ferramentas estão associadas com a tecnologia Java. Claro que existem várias outras que podem se aplicadas a demais tecnologias. O importante é entender a finalidade das mesmas e procurar a que mais se adequa ao seu processo de produção de software.

Uma outra questão importante sobre ferramentas de qualidade é que, sempre que possível, elas devem ser automatizadas. Qualidade em geral envolve muitas tarefas repetitivas que exigem uma grande demanda de pessoas qualificadas que se chateiam rapidamente ao executar a mesma tarefa várias vezes. Neste caso, automatize tudo o que for possível.

As equipes de testes de software expandirão significativamente os seus esforços de automação de testes no próximo ano, segundo uma pesquisa da TechWell que ouviu profissionais que atuam em testes de software e que são tomadores de decisões de QA (*Quality Assurance*) em empresas de médio e grande porte. Se você trabalha com desenvolvimento de software, sem dúvida, já ouviu falar bastante sobre automação de testes, que nos últimos anos tem tido a sua popularidade aumentada graças ao uso de ferramentas que facilitam esta tarefa e que permitem registrar os resultados deste trabalho. Esta popularidade da automação de testes acompanha o crescimento das áreas de nas áreas de DevOps¹⁰ e desenvolvimento Ágil nas companhias.

Essas três tendências - esforços para automação, uso de ferramentas e de metodologias Ágil e DevOps - caminham juntas, segundo a mesma pesquisa, que também descobriu que 72% das organizações estão atualmente automatizando os testes, sendo que destas, 76% adotaram metodologias ágeis e outras 55% estão usando as práticas DevOps. As empresas estão adotando essas práticas devido a mudanças no mercado e as expectativas dos usuários. A maioria delas utiliza o software como principal meio de interação com seus clientes e, para atender às expectativas, devem ser mais proativas em relação ao que os usuários querem e precisam. Isso significa desenvolver, testar e liberar aplicações de negócios com muito mais frequência e com a maior agilidade possível. A concorrência global este tipo de ação imediata.

Somado a esses fatores, tem havido também muitas pesquisas mostrando o impacto positivo que a entrega contínua de software tem sobre a as organizações com o objetivo de alcançar essas metas, mas nem tanto na área de automação de testes, um componente essencial para manter a alta qualidade em ambientes de entrega contínua. Os benefícios implementação da automação de testes são claros

Apesquisa da TechWell descobriu que as organizações que automatizam 50% ou mais de seus testes experimentam ciclos de testes mais rápidos (88%), enquanto que 71% apontou melhor cobertura de testes. Outras 68% melhoraram sua capacidade de detectar bugs antecipadamente. Isso significa um time-to-market mais rápido sem sacrificar a qualidade. Outros benefícios importantes incluem maior eficiência nos testes e melhoria na autoestima de toda a equipe de QA. Em essência, as pessoas adoram quando podem ser produtivos e entregar bens e serviços excelentes, o trabalho se torna divertido e significativo.

¹⁰ Sigla para a união de desenvolvimento de software com a operação

Ainda assim, a automação de testes é uma prática pouco utilizada nas empresas. Embora a maioria das organizações pesquisadas esteja automatizando pelo menos alguns testes, a maioria (76%) está automatizando menos da metade de todos os testes. Isso sinaliza a existência de alguns desafios na transição do teste manual para o teste automatizado conforme a seguir:

- A implementação da automação de testes exige que as equipes de controle de qualidade desenvolvam novas estratégias de teste, implementem novas ferramentas e aprendam novas habilidades
- Existem novos processos e fluxos de trabalho que afetam os testadores e outros membros importantes da equipe, incluindo desenvolvedores e analistas de negócios
- As organizações podem precisar contratar engenheiros de software capacitados em automação de testes que possam atingir o chão e liderar outros
- Tudo isso demanda tempo, dinheiro e paciência

As pessoas entrevistadas afirmaram que suas empresas lutam para criar equipes com o conjunto de habilidades apropriado, além de buscar ganhar a adesão da organização para as mudanças, justamente porque os testadores devem trabalhar mais rápido e necessitam atuar em colaboração contínua com outras áreas. Alguns testadores podem, também, se ressentir ou temer a necessidade de se tornarem mais técnicos em seus trabalhos.

A adoção de tecnologia também desempenha um papel significativo na automação de testes. Novas ferramentas de teste e estruturas que suportam práticas de automação de testes e DevOps complementares, como a integração contínua, são necessárias. Isso pode introduzir novas e mais complexidades para as equipes tradicionais de controle de qualidade. Existem dezenas de ferramentas e plataformas disponíveis hoje, tanto de código aberto quanto comercial. A escolha pode ser acertada, mas a seleção pode ser confusa. Muitas vezes, as organizações acabam trabalhando com muitas ferramentas repetitivas. Quase um terço das organizações pesquisadas afirma que é difícil acompanhar o trabalho de automação em várias ferramentas. Aproximadamente a mesma porcentagem expressou dificuldade em encontrar ferramentas que atendam às suas necessidades.

Além de selecionar as melhores ferramentas para o trabalho, os testadores de software também precisam ter liderança - e isso geralmente significa combater expectativas irreais e estabelecer novas. Equívocos comuns incluem que a automação de testes é rápida e fácil de implementar, que exigirá menos recursos do que testes manuais e que é gratuita ou tem custos mínimos.

A verdade é que é preciso tempo para construir uma estratégia de automação de testes e habilidade para escrever os scripts que a irão capacitar. E, embora existam muitas ferramentas eficazes de código aberto, recursos serão necessários para implementá-las e mantê-las. É melhor esclarecer isso antecipadamente, definir os cronogramas com base no que é realista e garantir que os recursos necessários sejam reservados e alocados para o projeto.

Estas são as dificuldades que surgem, existentes em qualquer grande transformação no

local de trabalho. A vantagem é que as organizações que obtiverem sucesso na automação de testes e expandirem esta prática com o tempo serão capazes de oferecer mais valor aos negócios e aproveitar melhor seu tempo.

Muitos dos testes manuais que a automação substitui são aqueles que são repetidos várias vezes. A automação reduz os erros que podem ocorrer em testes repetitivos e evita que os profissionais fiquem entediados. A automação permite que as equipes dediquem mais tempo aos testes mais criativos e à resolução de problemas, preencher as lacunas e oferecer uma experiência de usuário realmente incrível.

Os entrevistados pela TechWell também afirmaram que a automação permite que eles gastem seu tempo de forma mais estratégica - planejando e executando planos melhores de automação e realizando testes exploratórios, que não podem ser substituídos pela automação. Um dos respondentes resumiu da seguinte forma:

"Não usamos a automação para reduzir os testes manuais; a usamos para aumentá-los e permitir que nossos testadores humanos se concentrem em novas funcionalidades e trabalhos de testes de alto valor".

As equipes de QA devem ver a automação como uma atividade complementar ao teste manual versus a substituição de insight inestimável por ferramentas. Quase 50% dos entrevistados afirmou que esperam automatizar 50% ou mais de todos os testes daqui a um ano. Preparar-se cedo para esta nova etapa pode capacitar a sua equipe para menos dores e a obter o sucesso de maneira mais rápida. Se a sua organização ainda não está automatizando os testes, provavelmente - acreditamos - que ela estará em breve.

Considerações Finais

Neste estudo abordamos os aspectos da operacionalização da qualidade de software. O importante aqui não é somente entender as ferramentas, mas sim entender a finalidade das mesmas, permitindo a aplicação destas ao seu processo de software. Pode-se dizer que esses são os primeiros benefícios de se investir em entrega contínua. Obviamente, existem outras vantagens, como a agilização de processos, a melhoria da performance da equipe, maior segurança e qualidade nas entregas, aumento da satisfação dos clientes.

Agilização dos processos

Um dos principais objetivos da entrega contínua é aumentar a velocidade nas entregas da equipe, garantindo a qualidade do produto final. Sendo assim, é possível dizer que a entrega contínua busca incessantemente o denominador final entre a maneira mais ágil de produção e o

melhor atendimento para os clientes.

Melhoria da performance da equipe

A entrega contínua propicia a integração de todos os envolvidos no processo produtivo, permitindo que toda a equipe esteja alinhada e trabalhe em equipe para conquistar os objetivos determinados. Todos os colaboradores se tornarão mais eficientes, por meio de participações pontuais, mas efetivas, dinâmicas e diretas. Além disso, o conhecimento adquirido para a implementação da entrega contínua será valioso para a prestação de seus serviços.

Maior segurança e qualidade nas entregas

A automatização dos processos será fundamental para prover confiança na qualidade e nas alterações que serão realizadas. É importante lembrar que a falha humana é a principal responsável pela ocorrência de problemas no código (problemas que não existem em fluxos de entrega contínua).

Aumento da satisfação dos clientes

O último benefício que elencamos está ligado à satisfação dos clientes. Imagine que seja encontrado um problema em um projeto que já esteja em ambiente de produção. Caso isso aconteça, será possível acionar a equipe e os desenvolvedores.

Assim, as modificações e correções poderão ser implementadas de maneira rápida e eficiente. O software entregue estará de acordo com as necessidades do cliente, evitando que ele tenha que esperar um longo tempo por uma atualização.

Referências

PRESSMAN, R.S., Engenharia de Software, Rio de Janeiro: McGraw Hill, Tradução da 6a edição, 2011.

PRESSMAN, R.S., Software Engineering: A Practitioner's Approach, McGraw Hill, 6 th edition, 2011.

SOMMERVILLE, I., Engenharia de Software, São Paulo: Addison-Wesley, 9a edição, 2011.

CHRISTENSEN, M.J., THAYER, R.H., The Project Manager's Guide to Software Engineering Best Practices, Wiley-IEEE Computer Society Press, 2002.

ABNT, NBR ISO 9000 – Sistemas de Gestão da Qualidade, 2000.

ABNT , NBR ISO/IEC 12207 – Tecnologia da Informação – Processos de Ciclo de Vida, 1998, Emenda 1: 2002, Emenda 2: 2004.

ISO/IEC 12207 (2008). Systems and Software Engineering 7 Software Life Cycle Process. International Organization for Standardization and the International Electrotechnical Commission. Geneva, Switzerland.

ISO/IEC 15504 – Information Technology - Process Assessment, 2003/2004.

CHRISSIS, M.B., KONRAD, M., SHRUM, S., CMMI: Guidelines for Process Integration and Product Improvement, 2 nd Ed., Addison Wesley, 2006.

SOFTEX, MPS.BR – Melhoria de Processo do Software Brasileiro – Guia Geral, 2009.

BOEHM, B.W., ABTS, C., BROWN, A.W., CHULANI, S., CLARK, B.K., HOROWITZ, E., MADACHY, R., REIFER, D., STEECE, B., Software Cost Estimation with COCOMO II, Prentice Hall, 2000.

PMI. Um guia do conhecimento em gerenciamento de projetos. Guia PMBOK® 6a. ed. – EUA: Project Management Institute, 2017.